

```
(<expression1> + <expression2>) == add(<expression1>,<expression2>)
(<expression1> - <expression2>) == diff(<expression1>,<expression2>)
(<expression1> * <expression2>) == mult(<expression1>,<expression2>)
(<expression1> / <expression2>) == quot(<expression1>,<expression2>)
<integer> == numb(<integer>)
```

For example:

```
1 == numb(1)
(1 + 2) == add(numb(1),numb(2))
((1 * 2) + 3) == add(mult(numb(1),numb(2)),numb(3))
((1 * 2) + (3 - 4)) == add(mult(numb(1),numb(2)),diff(numb(3),numb(4)))
```

Write a function which evaluates an arithmetic expression in this representation, for example:

```
- eval (numb(1));
> 1 : exp

- eval (add(numb(1),numb(2)));
> 3 : exp

- eval (add(mult(numb(1),numb(2)),numb(3)));
> 5 : exp

- eval (add(mult(numb(1),numb(2)),diff(numb(3),numb(4))));
> 1 : exp
```

10. FUNCTIONAL PROGRAMMING AND LISP

10.1. Introduction

LISP(LISt Processor) is a widely used artificial intelligence language. It is weakly typed with run-time type checking. Functions calls are evaluated in applicative order. LISP lacks structure and pattern matching.

Although LISP is not a pure functional language, it has a number of functional features. Like the λ calculus, LISP has an incredibly simple core syntax. This is based on bracketed symbol sequences which may be interpreted as list data structures or as function calls. The shared representation of data and program reputedly makes LISP particularly appropriate for artificial intelligence applications.

The original LISP programming system was devised by McCarthy in the late 1950's as an aid to the Advice Taker experimental artificial intelligence system. McCarthy's early description of LISP was based on a functional formalism influenced by λ calculus, known as **M-expressions(Meta expressions)**. These were represented in an extremely simple **S-expression'(Symbolic expression)** format for practical programming. Contemporary LISP systems are based solely on the S-expression format although other functional languages are reminiscent of the richer M-expressions. We won't consider M-expressions here.

LISP, like BASIC, is not a unitary language and is available in a number of widely differing dialects. The heart of these differences, as we shall see, lies in the way that name/object associations are treated. Here we will consider COMMON LISP which is a modern standard. We will also look briefly at Scheme, a fully functional LISP. Other LISPs include FRANZ LISP which is widely available on UNIX systems, MACLISP and INTERLISP which COMMON LISP subsumes, and Lispkit Lisp which is another fully functional LISP.

It is important to note that LISP has been refined and developed for many years and so is not a very 'clean' language. LISP systems differ in how some aspects of LISP are implemented. Some aspects of LISP are extremely arcane and subject to much disputation amongst the cognoscenti. Furthermore, while LISP has much in common with functional

languages it is actually used as an imperative programming language for most applications.

We will only look at enough LISP to see how it corresponds to our functional approach. Many details will, necessarily, be omitted.

10.2. Atoms, numbers and symbols

The basic LISP objects are **atoms** composed of sequences of printing characters. Whenever a LISP system sees an atom it tries to evaluate it.

COMMON LISP provides distinct representations for integer, ratio, floating point and complex **number** atoms. Here, we will only consider integers. These consist of digit sequences preceded by an optional sign, for example:

0 42 -99

The result of evaluating a number is that number.

Symbols or **literals** are non-numeric atoms and correspond to names, for example:

banana BANANA forty_two --> +

Symbols have associated values. The result of evaluating a symbol is its associated value. There are a large number of system symbols with standard associated values known as **primitives**. As we will see, symbols are also objects in their own right.

10.3. Forms, expressions and function applications

The **form** is the basic LISP construct and consists of an atom or a left bracket followed by zero or more atoms or forms ending with a right bracket:

<form> ::= <atom> | (<forms>) | ()
<forms> ::= <form> | <form> <forms>

Forms are used for all expressions and data structures in LISP. Forms are always strictly bracketed except when special shorthand constructs are introduced.

Expressions are always prefix. The first form in a bracketed form sequence is interpreted as the function. Subsequent forms are interpreted as arguments. Thus, in a bracketed sequence of forms, we will refer to the first form as the **function** and to the subsequent forms as the **arguments**. We will also refer to primitives as if they were system functions.

Forms are evaluated in applicative order from left to right. For expressions, we will use -> to indicate a result after applicative order evaluation.

Note that the function form may be the name of a function or a lambda expression but may NOT be an expression returning a function!

Note that the argument form may NOT be a lambda function or the name from a global definition or the name of a primitive!

Special primitives and techniques are used to treat functions as values.

10.4. Logic

`t`

is the primitive for TRUE and:

`nil`

is the primitive for FALSE.

`not and or`

are the primitives for the logical negation, conjunction and disjunction functions respectively. These may be used to construct simple logical expressions as forms, for example:

```
(not t) ->  
nil
```

```
(and t nil) ->  
nil
```

```
(or (and t nil) (not nil)) ->  
t
```

In LISP, unlike most programming languages, `and` and `or` may have more than two arguments. For `and`, the final value is the conjunction of all the arguments, for example:

```
(and t nil t) ->  
nil
```

For `or`, the final value is the disjunction of all the arguments, for example:

```
(or t nil t) ->  
t
```

10.5. Arithmetic and numeric comparison

`+ - * /`

are the primitives for the addition, subtraction, multiplication and division functions. These may be used with numbers to construct simple arithmetic expressions, for example:

```
(+ 40 2) ->  
42
```

```
(- 46 4) ->  
42
```

```
(* 6 (+ 3 4)) ->  
42
```

```
(/ (+ 153 15) (- 7 3)) ->  
42
```

As with `and` and `or`, these functions may have more than two arguments, so `+` returns the sum, `-` the difference, `*` the product and `/` the overall quotient, for example:

```
(+ 12 25 5) ->  
42
```

```
(- 59 8 9) ->  
42
```

```
(* 3 2 7) ->  
42
```

```
(/ 336 4 2) ->  
42
```

/ is actually a real division operator. The primitive

`truncate`

rounds a single argument down to the nearest integer. If `truncate` is given two arguments then it divides one by the other and rounds the result down to the nearest integer, for example:

```
(truncate 43 6) ->  
7
```

The primitive:

`rem`

returns the integer remainder after division, for example:

```
(rem 43 6) ->  
1
```

The numeric less than, less than or equal, equality, greater than and greater than or equal primitive comparison functions are:

`< <= = >= >`

These may all be used with more than two arguments. Thus `=` checks that all its arguments are equal, for example:

```
(= 2 2 2 2 2) ->  
t
```

`<=` checks that its arguments are in ascending order, for example:

```
(<= 1 2 2 3 4 5) ->  
t
```

`<` checks that its arguments are in strictly ascending order, for example:

```
(< 1 2 2 3 4 5) ->  
nil
```

`>=` checks that its arguments are in descending order, for example:

```
(>= 5 4 3 3 2 1) ->  
t
```

and `>` checks that its arguments are in strictly descending order, for example:

```
(> 9 8 7 6 5) ->  
t
```

The primitive:

```
numberp
```

returns true if its argument is a number. For example,

```
(numberp 42) ->  
t
```

10.6. Lambda functions

LISP uses a notation like that for the λ calculus to define nameless functions.

It is important to note that LISP functions do not have all the properties we might expect from the λ calculus. In particular, special techniques are needed to pass functions as arguments, to return functions as values and to apply functions returned as values to new arguments.

Functions are defined as forms with the primitive:

```
lambda
```

followed by a flat list of bound variables and the body form:

```
(lambda (<bound variables>) <body>)
```

where:

```
<bound variables> ::= <bound variable> | <bound variable> <bound variables>
```

For example, to square a number:

```
(lambda (x) (* x x))
```

or to find the sum of the squares of two numbers:

```
(lambda (x y) (+ (* x x) (* y y)))
```

or to find the value of the quadratic:

$$ax^2+bx+c$$

given a , b , c and x :

```
(lambda (a b c x) (+ (* a (* x x)) (* b x) c)))
```

Note that functions are normally uncurried in LISP.

Note that LISP systems will reject attempts to present lambda functions directly as values other than in the function position in a form. `lambda` is not a primitive which denotes a system function. Instead it acts as a marker to indicate a function form. However, if a LISP system sees a naked lambda function form it will try to find a function associated with `lambda` and fail. The special techniques needed to manipulate function values are discussed below.

A function is applied to arguments in a form with the function followed by the arguments. The function's body is evaluated with the bound variables associated initially with the corresponding arguments:

(<function> <argument1> <argument2> ...)

Note that arguments for uncurried functions are not bracketed but follow straight after the function.

For example:

```
((lambda (x) (* x x)) 2) ->  
4
```

```
((lambda (x y) (+ (* x x) (* y y))) 3 4) ->  
25
```

```
((lambda (a b c x) (+ (* a (* x x)) (* b x) c))) 1 2 1 1) ->  
4
```

10.7. Global definitions

LISP systems are usually interactive and accept forms from the input for immediate evaluation. Global definitions provide a way of naming functions for use in subsequent forms.

A definition is a form with the primitive:

`defun`

followed by the name, bound variable list and the body form:

```
(defun <name> (<bound variable list>) <body>)
```

Many LISP systems print the defined name after a definition. For example:

```
(defun sq (x) (* x x)) ->  
sq
```

```
(defun sum_sq (x y) (+ (* x x) (* y y))) ->  
sum_sq
```

```
(defun quad (a b c x) (+ (* a (* x x)) (* b x) c)) ->  
quad
```

A defined name may then be used instead of a lambda function in other forms. For example:

```
(sq 2) ->  
4
```

```
(sum_sq 3 4) ->  
25
```

```
(quad 1 2 1 2) ->  
9
```

In particular, defined names may be used in other definitions. For example, the last two definitions might be shortened to:

```
(defun sum_sq (x y) (+ (sq x) (sq y)))
```

```
(defun quad (a b c x) (+ (* a (sq x)) (* b x) c))
```

It is important to note that a global definition establishes a special sort of name/value relationship which is not the same as that between bound variables and values.

10.8. Conditional expressions

LISP conditional expressions are forms with the primitive:

`cond`

followed by a sequence of options. Each option is a test expression followed by a result expression which is to be evaluated if the test expression is true:

```
(cond (<test1> <result1>)
      (<test2> <result2>)
      ...
      (t <resultN>))
```

Note that the last option's test expression is usually `t` to ensure a final value for the conditional.

When a conditional expression is evaluated, each option's test expression is tried in turn. When a true test expression is found, the value of the corresponding result expression is returned. This is like a nested sequence of `if ... then ... else` expressions.

Note that a conditional expression is not evaluated in applicative order.

For example, to find the larger of two values:

```
(defun max (x y)
  (cond ((> x y) x)
        (t y)))
```

or to define logical `not`:

```
(defun lnot (x)
  (cond (x nil)
        (t t)))
```

or to define logical `and`:

```
(defun land (x y)
  (cond (x y)
        (t nil)))
```

COMMON LISP provides a simpler conditional primitive:

`if`

which is followed by a test and expressions to be evaluated if the test is true or false:

```
(if <test>
    <true result>
    <false result>)
```

For example, to find the smaller of two numbers:

```
(defun min (x y)
  (if (< x y)
```

```
x  
y))
```

or to define logical or :

```
(defun lor (x y)  
  (if x  
      t  
      y))
```

10.9. Quoting

We said above that in LISP forms are used as both program and data structures. So far, we have used forms as function calls in general and to build higher level control structures using special system primitives which control form evaluation. In order to use forms as data structures additional primitives are used to prevent form evaluation. This is a different approach to the λ calculus where data structures are packaged as functions with bound variables to control the subsequent application of selector functions.

In LISP, a mechanism known as **quoting** is used to delay form evaluation. This is based on the idea from ordinary language use that if you want to refer to something's name rather than that thing then you put the name in quotation marks. For example:

Edinburgh is in Scotland

is a statement about a city, whereas:

'Edinburgh' has nine letters

is a statement about a word. Putting in the quotation marks shows that we are interested in the letter sequence rather than the thing which the letter sequence refers to. In LISP, quoting is used to prevent form evaluation. Quoting a form shows that we are interested in the sequence of sub-forms as a structure rather than the form's final value. Quoting is a special sort of abstraction for delaying evaluation. Later on, we will see how it can be reversed.

When the LISP primitive:

quote

is applied to an argument then that argument is returned unevaluated:

```
(quote <argument>) -> <argument>
```

In particular, a symbol argument is not replaced by its associated value but becomes an object in its own right.

Quoting is so widely used in LISP that the special notation:

```
'<argument>
```

has been introduced. This is equivalent to the above use of the `quote` primitive.

10.10. Lists

LISP is perhaps best known for the use of list processing as the basis of programming.

The empty list is the primitive:

`nil`

which may be also written as:

`()`

The tester for an empty list is the primitive:

`null`

This is the same as `not` because `FALSE` is `nil` in LISP and anything which is not `FALSE` is actually `TRUE`!

Lists may be constructed explicitly with the:

`cons`

primitive:

`(cons <head> <tail>)`

The `<head>` and `<tail>` arguments are evaluated, and a list with `<head>`'s value in the head and `<tail>`'s value in the tail is formed.

If the eventual tail is not the empty list then the **dotted pair** representation is used for the resultant list:

`<head value> . <tail value>`

For example:

```
(cons 1 2) ->  
1 . 2
```

```
(cons 1 (cons 2 3)) ->  
1. (2 . 3)
```

```
(cons (cons 1 2) (cons 3 4)) ->  
(1 . 2) . (3 . 4)
```

```
(cons (cons 1 2) (cons (cons 3 4) (cons 5 6))) ->  
(1 . 2) . ((3 . 4) . (5 . 6))
```

When a list ends with the empty list then a flat representation based on forms is used with an implicit empty list at the end. For example:

```
(cons 1 nil) ->  
(1)
```

```
(cons 1 (cons 2 nil)) ->  
(1 2)
```

```
(cons (cons 1 (cons 2 nil)) (cons (cons 3 (cons 4 nil)) nil)) ->  
((1 2) (3 4))
```

Thus, as in chapter 6, the empty list `nil` is equivalent to the empty form `()`.

Note that lists built by `cons` and ending with the empty list appear to be forms but are not actually evaluated further as function calls.

Lists may be constructed directly in form notation and this is the most common approach. Note, however, that list forms must be explicitly quoted to prevent function call evaluation. For example:

```
(1 2 3)
```

looks like a call to the function 1 with arguments 2 and 3, whereas:

```
'(1 2 3)
```

is the list:

```
1 . (2 . (3 . nil))
```

The primitive:

```
list
```

is a multi-argument version of `cons` but constructs a list ending with the empty list. For example:

```
(list 1 2) ->  
(1 2)
```

```
(list 1 2 3) ->  
(1 2 3)
```

```
(list (list 1 2) (list 3 4)) ->  
((1 2) (3 4))
```

```
(list (list 1 2) (list 3 4) (list 5 6)) ->  
((1 2) (3 4) (5 6))
```

The primitive:

```
listp
```

returns true if its argument is a list.

For example:

```
(listp '(1 2 3 4)) ->  
t
```

10.11. List selection

The head of a LISP list is known as the **car** and the tail as the **cdr**. This is from the original IBM 704 implementation where a list head was processed as the 'Contents of the Address Register' and the tail as the 'Contents of the Decrement Register'. Thus:

```
car
```

is the head selection primitive and:

```
cdr
```

is the tail selection primitive. For example:

```
(car '(1 2 3)) ->  
1
```

```
(cdr '(1 2 3)) ->  
(2 3)
```

```
(car (cdr '(1 2 3))) ->  
2
```

```
(cdr (cdr '(1 2 3))) ->  
(3)
```

Note that sub-lists selected from lists appear to be forms but are not further evaluated as function call forms by `car` or `cdr`.

10.12. Recursion

In LISP, recursive functions are based on function definitions with the defined name appearing in the function body. For example, to find the length of a linear list:

```
(defun length (l)  
  (if (null l)  
      0  
      (+ 1 (length (cdr l)))))
```

and to count how often a value appears in a linear list of numbers:

```
(defun count (x l)  
  (cond ((null l) 0)  
        ((= x (car l)) (+ 1 (count x (cdr l))))  
        (t (count x (cdr l)))))
```

For example, to insert a value into an ordered list:

```
(defun insert (x l)  
  (cond ((null l) (cons x nil))  
        ((< x (car l)) (cons x l))  
        (t (cons (car l) (insert x (cdr l))))))
```

and to sort a list:

```
(defun sort (l)  
  (if (null l)  
      nil  
      (t (insert (car l) (sort (cdr l))))))
```

10.13. Local definitions

LISP provides the:

```
let
```

primitive for the introduction of local variables in expressions:

```
(let ((<variable1> <value1>)
```

```
(<variable2> <value2>)  
...)  
(<result>))
```

is equivalent to the function call:

```
((lambda (<variable1> <variable2> ... ) <result>) <value1> <value2> ... )
```

For example, to insert a value into an ordered list if it is not there already:

```
(defun new_insert (x l)  
  (if (null l)  
      (cons x nil)  
      (let ((hl (car l))  
            (tl (cdr l)))  
        (cond ((= x hl) l)  
              ((< x hl) (cons x l))  
              (t (cons hl (new_insert x tl)))))))
```

Here, if the list is empty then a new list with the value in the head is returned. Otherwise, the head and tail of the list are selected. If the head matches the value then the list is returned. If the value comes before the head then it is added before the head. Otherwise the value is inserted in the tail.

10.14. Binary trees in LISP

In chapter 7 we looked at the construction of binary trees using a list representation. This translates directly into LISP: we will use `nil` for the empty tree and the list:

```
(<item> <left> <right>)
```

for the tree with node value `<item>`, left branch `<left>` and right branch `<right>`.

We will use:

```
(defun node (item left right) (list item left right))
```

to construct new nodes.

LISP has no pattern matching so it is useful to define selector functions:

```
(defun item (l) (car l))  
  
(defun left (l) (car (cdr l)))  
  
(defun right (l) (car (cdr (cdr l))))
```

for the node value, left branch and right branch.

Thus, to add an integer to an ordered binary tree:

```
(defun tadd (i tree)  
  (cond ((null tree) (node i nil nil))  
        ((< i (item tree)) (node (item tree) (tadd i (left tree) (right tree))))  
        (t (node (item tree) (left tree) (tadd i (right tree))))))
```

For example:

```
(tadd 7 nil) ->
(7 nil nil)

(tadd 4 '(7 nil nil)) ->
(7
 (4 nil nil)
 nil)

(tadd 10 '(7 (4 nil nil) nil)) ->
(7
 (4 nil nil)
 (10 nil nil))

(tadd 2 '(7 (4 nil nil) (10 nil nil))) ->
(7
 (4
  (2 nil nil)
  nil)
 (10 nil nil))

(tadd 5 '(7 (4 (2 nil nil) nil) (10 nil nil))) ->
(7
 (4
  (2 nil nil)
  (5 nil nil))
 (10 nil nil))
```

Hence, to add a list of numbers to an ordered binary tree:

```
(defun taddlist (l tree)
  (if (null l)
      tree
      (taddlist (cdr l) (tadd (car l) tree))))
```

Finally, to traverse a binary tree in ascending order:

```
(defun traverse (tree)
  (if (null tree)
      nil
      (append (traverse (left tree))
               (cons (item tree) (traverse (right tree))))))
```

For example:

```
(traverse '(7 (4 (2 nil nil) (5 nil nil) (10 nil nil)))) ->
(2 4 5 7 10)
```

10.15. Dynamic and lexical scope

LISP is often presented as if it were based on the λ calculus but this is somewhat misleading. LISP function abstraction uses a notation similar to the λ abstraction but the relationship between bound variables and variables in expressions is rather opaque.

In our presentation of λ calculus, names have **lexical** or **static** scope. That is, a name in an expression corresponds to the bound variable of the innermost enclosing function to define it.

Consider the following contrived example. We might define:

```
def double_second = λx.λx.(x + x)
```

This is a function with bound variable:

x

and body:

```
λx.(x + x)
```

Thus, in the expression:

```
(x + x)
```

the xs correspond to the second rather than the first bound variable. We would normally avoid any confusion by renaming:

```
def double_second = λx.λy.(y + y)
```

For lexical scope, the bound variable corresponding to a name in an expression is determined by their relative positions in that expression, before the expression is evaluated. Expression evaluation cannot change that static relationship.

Early LISPs were based on **dynamic** scope where names' values are determined when expressions are evaluated. Thus, a name in an expression corresponds to the most recent bound variable/value association with the same name, when that name is encountered during expression evaluation. This is effectively the same as lexical scope when a name is evaluated in the scope of the corresponding statically scoped bound variable.

However, LISP functions may contain free variables and a function may be created in one scope and evaluated in another. Thus, a name might refer to different variables depending on the scopes in which it is evaluated. Expression evaluation can change the name/bound variable correspondence.

For example, suppose we want to calculate the tax on a gross income but do not know the rate of tax. Instead of making the tax rate a bound variable we could make it a free variable. Our approach to the λ calculus does not allow this unless the free variable has been introduced by a previous definition. However, this is allowed in LISP:

```
(defun tax (gross)
  (/ (* gross rate) 100))
```

Note that **rate** is free in the function body. For a LISP with dynamic scope, like FRANZ LISP, **rate**'s value is determined when:

```
(/ (* gross rate) 100)
```

is evaluated. For example, if the lowest rate of tax is 25% then we might define:

```
(defun low_tax (gross)
  (let ((rate 25))
    (tax gross)))
```

When **low_tax** is applied to an argument, **rate** is set to 25 and then **tax** is called. Thus, **rate** in **tax** is evaluated in the scope of the local variable **rate** in **low_tax** and will have the value 25.

In LISPs with dynamic scope this use of free variables is seen as a positive advantage because it delays decisions about name/value associations. Thus, the same function with free variables may be used to different effect in different places.

For example, suppose the average tax rate is 30%. We might define:

```
(defun av_tax (gross)
  (let ((rate 30))
    (tax gross)))
```

Once again, the call to `tax` in `av_tax` evaluates the free variable `rate` in `tax` in the scope of the local `rate` in `av_tax`, this time with value 30.

It is not clear whether or not dynamic scope was a conscious feature or the result of early approaches to implementing LISP. COMMON LISP is based on lexical scope but provides a primitive to make dynamic scope explicit if it is needed. Attempts to move lexically scoped free variables in and out of different scopes are faulted in COMMON LISP.

10.16. Functions as values and arguments

In looking at the λ calculus, we have become used to treating functions as objects which can be manipulated freely. This approach to objects is actually quite uncommon in programming languages, in part because until comparatively recently it was thought that it was hard to implement.

In LISP, functions are not like other objects and cannot be simply passed around as in the λ calculus. Instead, function values must be identified explicitly and applied explicitly except in the special case of the function form in a function application form.

The provision of functions as **first class citizens** in LISP used to be known as the **FUNARG problem** because implementation problems centred on the use of functions with free variables as arguments to other functions. This arose because of dynamic scope where a free variable is associated with a value in the calling rather than the defining scope. However, it is often necessary to return a function value with free variables frozen to values from the defining scope. We have used this in defining typed functions in chapter 5.

The traditional way round the FUNARG problem is to identify function values explicitly so that free variables can be frozen in their defining scopes. The application of such function values is also made explicit so that free variables are not evaluated in the calling scope. This freezing of free variables in a function value is often implemented by constructing a **closure** which identifies the relationship between free and lexical bound variables.

COMMON LISP is based on lexical scope where names are frozen in their defining scope. None the less, COMMON LISP still requires function values to be identified and applied explicitly.

In COMMON LISP, the primitive:

```
function
```

is used as a special form of quoting for functions:

```
(function <function>)
```

It creates a function value in which free variables are associated with bound variables in the defining scope. As with `quote`, an equivalent special notation:

```
#'<function>
```

is provided.

Note that most LISP systems will not actually display function values as text. This is because they translate functions into intermediate forms to ease implementation but lose the equivalent text. Some systems may display an implementation dependent representation.

For example, we could define a general tax function:

```
(defun gen_tax (rate)
  #'(lambda (gross) (/ (* gross rate) 100)))
```

We might then produce the low and average tax rate functions as:

```
(gen_tax 25)
```

and

```
(gen_tax 30)
```

which return lambda function values with `rate` bound to 25 and 30 respectively.

The primitive:

```
funcall
```

is used to apply a function value to its arguments:

```
(funcall <function value> <argument1> <argument2> ...)
```

For example, to apply the low tax function to a gross income:

```
(funcall (gen_tax 25) 10000) ->
2500
```

This call builds a function value with `rate` bound to 25 and then applies it to the gross income 10000. Similarly:

```
(funcall (gen_tax 30) 15000) ->
4500
```

builds a function value with `rate` bound to 30 and then applies it to the gross income 15000.

For example, the mapping function `mapcar` may be defined as:

```
(defun mapcar (fn arg)
  (if (null arg)
      nil
      (cons (funcall fn (car arg)) (mapcar fn (cdr arg)))))
```

Note that the function `fn` is applied explicitly to the argument `car arg` by `funcall`. Now, we could square every element of a list with:

```
(defun sq_list (l)
  (mapcar #'(lambda (x) (* x x)) l))
```

Note that the function argument for `mapcar` has been quoted with `#'`.

Note that even if a function value argument is identified simply by name then that name must still be quoted with `#'` before it may be used as an argument. For example, to apply `Sq` to every element of a list:

```
(mapcar #'sq '(1 2 3 4 5))
```

10.17. Symbols, quoting and evaluation

Normally, symbols are variable names but they may also be used as objects in their own right. Quoting a symbol prevents the associated value being extracted. Thereafter, a quoted symbol may be passed around like any object.

One simple use for quoted symbols is as Pascal-like user defined enumeration types. For example, we might define the days of the week as:

```
'Monday 'Tuesday 'Wednesday 'Thursday 'Friday 'Saturday 'Sunday
```

We could then write functions to manipulate these objects. In particular, they may be compared with the equality primitive which is true if its arguments are identical objects:

eq

Note that in Pascal enumeration types are mapped onto integers and so have successor functions defined automatically for them. In LISP, we have to define a successor function explicitly if we need one. For example:

```
(defun next_day (day)
  (cond ((eq day 'Monday)    'Tuesday)
        ((eq day 'Tuesday)   'Wednesday)
        ((eq day 'Wednesday) 'Thursday)
        ((eq day 'Thursday)  'Friday)
        ((eq day 'Friday)    'Saturday)
        ((eq day 'Saturday)  'Sunday)
        ((eq day 'Sunday)    'Monday)))
```

The primitive:

eval

forces evaluation of a quoted form as a LISP expression. Thus, functions can construct quoted forms which are program structures for later evaluation by other functions. Thus, compiling techniques can be used to produce LISP from what is apparently data. For example, the rules for an expert system might be translated into functions to implement that system. Similarly, the grammar for an interface language might be used to generate functions to recognise that language.

For example, suppose we want to translate strictly bracketed infix arithmetic expressions:

```
<expression> ::= <number> |
                (<expression> + <expression>) |
                (<expression> - <expression>) |
                (<expression> * <expression>) |
                (<expression> / <expression>)
```

into prefix form so:

```
(<expression> + <expression>) == (+ <expression> <expression>)
(<expression> - <expression>) == (- <expression> <expression>)
(<expression> * <expression>) == (* <expression> <expression>)
(<expression> / <expression>) == (/ <expression> <expression>)
```

We need to extract the operator and place it at the head of a list with the translated expressions as arguments:

```
(defun trans (l)
  (if (numberp l)
      l
      (let ((e1 (trans (car l))))
```

```
(op (car (cdr l)))  
(e2 (trans (car (cdr (cdr l))))))  
(list op e1 e2)))
```

For example:

```
(trans '(1 + 2)) ->  
(+ 1 2)  
  
(trans '(3 * (4 * 5))) ->  
(* 3 (* 4 5))  
  
(trans '((6 * 7) + (8 - 9))) ->  
(+ (* 6 7) (- 8 9))
```

Note that quoted symbols for infix operators have been moved into the function positions in the resultant forms.

Now, we can evaluate the translated expression, as a LISP form, using `eval`:

```
(defun calc (l)  
  (eval (trans l)))
```

For example:

```
(calc '((6 * 7) + (8 - 9))) ->  
41
```

The treatment of free variables in quoted forms depends on the scope rules. For dynamic scope systems, quoted free variables are associated with the corresponding bound variable when the quoted form is evaluated. Thus, with dynamic scope, quoting can move free variables into different evaluation scopes.

In COMMON LISP, with lexical scope, quoted free variables are not associated with bound variables in the defining scope as might be expected. Instead, they are evaluated as if there were no bound variables defined apart from the names from global definitions.

10.18. λ calculus in LISP

We can use `function` and `funcall` for rather clumsy applicative order pure λ calculus. For example, we can try out some of the λ functions we met in chapter 2. First of all, we can apply the identity function:

```
#'(lambda (x) x)
```

to itself:

```
(funcall #'(lambda (x) x) #'(lambda (x) x))
```

Note that some LISP implementations will not print out the resultant function and others will print a system sepecific representation.

Let us now use definitions:

```
(defun identity (x) x) ->  
identity  
  
(funcall #'identity #'identity)
```

Note once again that there may be no resultant function or an internal representation will be printed.

Let us apply the self application function to the identity function:

```
(funcall #'(lambda (s) (funcall s s)) #'(lambda (x) x))
```

Notice that we must make the application of the argument to itself explicit. Now, let us use definitions:

```
(defun self_apply (s) (funcall s s)) ->
self_apply

(funcall #'self_apply #'identity)
```

Finally, let us try the function application function:

```
(defun apply (f a) (funcall f a)) ->
apply
```

with the self application function and the identity function:

```
(funcall #'apply #'self_apply #'identity)
```

Pure λ calculus in LISP is complicated by this need to make function values and their applications explicit and by the absence of uniform representations for function values.

10.19. λ calculus and Scheme

Scheme is a language in the LISP tradition which provides function values without explicit function identification and application. Scheme, like other LISPs, uses the bracket based form as the combined program and data structure, is weakly typed and has applicative order evaluation. Like COMMON LISP, Scheme is lexically scoped. We are not going to look at Scheme in any depth. Here, we are only going to consider the use of function values.

In Scheme, functions are true first class objects. They may be passed as arguments and function expressions may appear in the function position in forms. Thus, many of our pure λ calculus examples will run with a little translation into Scheme. However, Scheme systems may not display directly function values as text but may produce a system dependent representation.

Let us consider once again the examples from chapter 2. We can enter a lambda function directly, for example the identity function:

```
(lambda (x) x)
```

We can also directly apply functions to functions, for example we might apply the identity function to itself:

```
((lambda (x) x) (lambda (x) x))
```

Alas, the representation of the result depends on the system.

Scheme function definitions have the form:

```
(define (<name> <argument1> <argument2> ...) <body>)
```

Defined functions may be applied without quoting:

```
(define (identity x) x) ->
identity
```

```
(identity identity)
```

Note once again that there may be no resultant function or a system specific internal representation may be printed.

To continue with the self application and function application functions:

```
(lambda (s) (s s))  
  
((lambda (s) (s s)) (lambda (x) x))  
  
(define (self_apply s) (s s)) ->  
self_apply  
  
(self_apply identity)  
  
(define (apply f a) (f a)) ->  
apply  
  
(apply self_apply identity)
```

This explicit use of function values is much closer to the λ calculus than COMMON LISP though there is still no uniform representation for function values.

10.20. Other features

It is impossible to consider an entire language in a small chapter. Here, we have concentrated COMMON LISP in relation to pure functional programming. COMMON LISP has come along way since the original LISP systems. In particular, it includes a wide variety of data types which we have not considered including characters, arrays, strings, structures and multiple value objects. We also have not looked at input/output and other system interface facilities.

10.21. Summary

In this chapter we have:

- surveyed quickly COMMON LISP
- seen how to implement algorithms from preceding chapters in COMMON LISP
- seen that treating functions as objects in COMMON LISP involves explicit notation and that there is no standard representation for function results.
- seen that Scheme simplifies treating functions as objects but lacks a standard representation for function results.

10.22. Exercises

1) (c.f. Chapter 4 Exercises 1, 2, 3 & 4) Write functions to:

- i) find the sum of the integers between n and 0 .
- ii) find the product of the numbers between n and 1 .
- iii) find the sum of applying a function fun to the numbers between n and 0 .

iv) find the sum of applying a function `fun` to the numbers between `n` and `zero` in steps of `s`.

2) (c.f. Chapter 6 Exercise 2)

i) Write a function which indicates whether or not a list starts with a sub-list. For example:

```
(lstarts '(1 2 3) '(1 2 3 4 5)) ->
t
```

```
(lstarts '(1 2 3) '(4 5 6)) ->
nil
```

ii) Write a function which indicates whether or not a list contains a given sub-list. For example:

```
(lcontains '(4 5 6) '(1 2 3 4 5 6 7 8 9)) ->
t
```

```
(lcontains '(4 5 6) '(2 4 6 8 10)) ->
nil
```

iii) Write a function which counts how often a sub-list appears in another list. For example:

```
(lcount '(1 2) '(1 2 3 1 2 3 1 2 3)) ->
3
```

iv) Write a function which removes a sub-list from the start of a list, assuming that you know that the sub-list starts the list. For example:

```
(lremove '(1 2 3) '(1 2 3 4 5 6 7 8 9)) ->
(4 5 6 7 8 9)
```

v) Write a function which deletes the first occurrence of a sub-list in another list. For example:

```
(ldelete '(4 5 6) '(1 2 3 4 5 6 7 8 9)) ->
(1 2 3 7 8 9)
```

```
(ldelete '(4 5 6) '(2 4 6 8 10)) ->
(2 4 6 8 10)
```

vi) Write a function which inserts a sub-list after the first occurrence of another sub-list in a list. For example:

```
(linsert '(4 5 6) '(1 2 3) '(1 2 3 7 8 9)) ->
(1 2 3 4 5 6 7 8 9)
```

```
(linsert '(4 5 6) '(1 2 3) '(2 4 6 8 10)) ->
(2 4 6 8 10)
```

vii) Write a function which replaces a sub-list with another sub-list in a list. For example:

```
(lreplace '(6 5 4) '(4 5 6) '(9 8 7 6 5 4 3 2 1)) ->
(9 8 7 4 5 6 3 2 1)
```

```
(lreplace '(6 5 4) '(4 5 6) '(2 4 6 8 10)) ->
(2 4 6 8 10)
```

3) (c.f. Chapter 6 Exercise 3)

- i) Write a function which merges two ordered lists to produce an ordered list.
- ii) Write a function which merges a list of ordered lists.

4) (c.f. Chapter 7 Exercise 1)

The time of day might be represented as a list with three integer fields for hours, minutes and seconds:

(<hours> <minutes> <seconds>)

For example:

(17 35 42) == 17 hours 35 minutes 42 seconds

Note that:

24 hours = 0 hours
1 hour == 60 minutes
1 minute == 60 seconds

- i) Write functions to convert from a time of day to seconds and from seconds to a time of day. For example:

(too_secs (2 30 25)) ->
9025

(from_secs 48975) ->
(13 36 15)

- ii) Write a function which increments the time of day by one second. For example:

(tick (15 27 18)) ->
(15 27 19)

(tick (15 44 59)) ->
(15 45 0)

(tick (15 59 59)) ->
(16 0 0)

(tick (23 59 59)) ->
(0 0 0)

- iii) In a shop, each transaction at a cash register is time stamped. Given a list of transaction details, where each is a string followed by a time of day, write a function which sorts them into ascending time order. For example:

(tsort '((haggis (12 19 57))
 (bannocks (18 22 48))
 (white_pudding (10 12 35))
 (oatcakes (15 47 19)))) ->
((white_pudding (10 12 35))
 (haggis (12 19 57))
 (oatcakes (15 47 19))
 (bannocks (18 22 48)))

5) (c.f. Chapter 7 Exercise 2)

- i) Write a function which compares two integer binary trees.

- ii) Write a function which indicates whether or not one integer binary tree contains another as a sub-tree.
- iii) Write a function which traverses a binary tree to produce a list of node values in descending order.

Bibliography

This is an introductory book which draws on a wide range of sources. Most of the material here is covered in other texts though often with different perspectives and emphases. This bibliography is not exhaustive: rather, it offers a broad cross section of source and supplementary material. References are almost entirely to books on the grounds that these tend to be more discursive than academic papers.

Chapter 1 - Introduction

Backus³ influential paper contains a critique of von Neumann computing and arguments for functional programming.

Brief motivational material on functional programming is contained in Sadler and Eisenbach⁴⁸, Glaser, Hankin and Till²¹, Henderson³⁰ and Henson³¹.

Brady⁹ provides an accessible introduction to the theory of computing. There are further accounts in a wide variety of mathematical logic texts, of which Kleene's³⁶ and Mendelson's³⁸ are still outstanding.

For related general computing topics not considered further in this book: the denotational approach to programming language semantics is covered by Gordon²², Schmidt⁴⁹, and Stoy⁵¹; program specification is covered by Cohen, Harwood and Jackson¹⁵, Gehani & McGettrick²⁰, Hayes²⁹, Jones³³, and more formally by Turski & Maibaum⁵⁴; program verification is covered by Backhouse², Gries²³, Manna³⁷, and less formally though thoroughly by Bornat⁷.

For related functional programming and language topics not considered further in this book: Eisenbach^{8,16,24} and Glaser, Hankin and Till²¹ contain overviews of implementation techniques; SECD machine implementations are discussed by Brady⁹, Burge¹⁰, Field and Harrison¹⁸, Henderson³⁰, Henson³¹, and Wegner⁵⁵; combinators and graph reduction are covered thoroughly by Field and Harrison¹⁸ and by Peyton-Jones⁴³, and, in less detail, by Henson³¹ and Revesz⁴⁶; Bird and Wadler⁶ cover verification; Field and Harrison¹⁸ cover program transformation and abstract interpretation; Henson³¹ covers verification and transformation; Harrison and Khoshnevisan²⁷ also discuss transformation.

For functional languages not considered further in this book: Field and Harrison¹⁸ is based on Hope and also contains brief discussion of Miranda, Lisp and FP; Peyton-Jones⁴³ is based on Miranda and contains an introduction by Turner⁵²; Bailey⁴ covers Hope; Harrison and Khoshnevisan²⁸ cover FP; Henson³¹ covers FP; Glaser, Hankin and Till²¹ discuss briefly FP, Hope and KRC; Bird and Wadler contains a brief appendix on Miranda⁶; Revesz⁴⁶ discusses briefly FP and Miranda; SASL is covered by Turner⁵³.

For various imperative languages mentioned here: ALGOL 60⁴⁰; Algol 68⁴¹; BCPL⁴⁷; C³⁵; Pascal¹⁹; POP-2¹¹; Prolog¹⁴ and PS-algol¹²

Chapter 2 - λ calculus

Church's¹³ description of the λ calculus is much referenced but little read. Barendregt⁵ is the standard reference. Hindley and Seldin³² is less detailed.

Early descriptions from a computing perspective include Burge¹⁰, and Wegner⁵⁵. Field and Harrison¹⁸, Peyton-Jones⁴³ and Revesz⁴⁶ provide thorough contemporary accounts, as does Stoy⁵¹ though oriented to semantics. Glaser, Hankin and Till²¹, and Henson³¹ also describe λ calculus.

Pair functions are discussed by Barendregt⁵, Field and Harrison¹⁸, Glaser, Hankin and Till²¹, Henson³¹, Revesz⁴⁶ and Wegner⁵⁵.

Chapter 3 - Conditions, booleans and integers

Burge¹⁰, Field and Harrison¹⁸, Glaser, Hankin and Till²¹, Henson³¹, Revesz⁴⁶ and Wegner⁵⁵ all have accounts of aspects of the material on conditional expressions and booleans. Schmidt⁴⁹ and Stoy⁵¹ cover it in exercises.

There are many approaches to representing numbers. The approach here is discussed by Barendregt⁵, and Glaser, Hankin and Till²¹, and also by Revesz⁴⁶ in an exercise. Field and Harrison¹⁸ and Wegner⁵⁵ discuss variants. Church's representation is discussed by Barendregt⁵, Burge¹⁰, Henson³¹, Revesz⁴⁶, and Wegner⁵⁵.

Chapter 4 - Recursion

Barendregt⁵, Brady⁹, Glaser, Hankin and Till²¹, Henson³¹, Peyton-Jones⁴³, Revesz⁴⁶ and Stoy⁵¹ all discuss the derivation of the 'recursion' function. Field and Harrison¹⁸ present it. Schmidt⁴⁹ presents it in an exercise. Burge¹⁰ discusses it in terms of combinators.

Kleene³⁶ Mendelson³⁸, Peter⁴², and Rayward-Smith⁴⁵ provide accounts of the construction of arithmetic and comparison operations within recursive function theory. Brief computing oriented accounts are in Burge¹⁰ and Glaser, Hankin and Till²¹.

Chapter 5 - Types

The approach to types considered here is a λ calculus implementation of run-time typing with tags. Abelson and Sussman¹ discuss a related Scheme approach to manifest types, and Henderson³⁰ and Peyton-Jones⁴³ discuss implementations of typing.

Field and Harrison¹⁸ and Peyton-Jones⁴³ contain thorough accounts of type checking.

Chapter 6 - Lists and strings

Bird and Wadler⁶, Henderson³⁰, Henson³¹ and Revesz⁴⁶ contain thorough accounts of lists from a functional programming perspective.

Numerous books on LISP contain material on list processing and mapping functions. Wilensky⁵⁷ provides an accessible introduction.

Chapter 7 - Composite values and trees

The use of lists to represent composite values is discussed implicitly in numerous books on LISP.

Henderson³⁰ and Queinnec⁴⁴ discuss accumulation variables.

Abelson and Sussman¹ and Shapiro⁵⁰ discuss the list representation of binary trees in Scheme and LISP respectively.

Bird and Wadler⁶ provides a thorough account of trees from a functional language perspective.

Chapter 8 - Evaluation

Barendregt⁵ and Hindley and Seldin³² contain formal details of reduction. There are more accessible approaches in Brady⁹, Burge¹⁰, Field and Harrison¹⁸, Glaser, Hankin and Till²¹, Henson³¹, Peyton-Jones⁴³, Revesz⁴⁶, Stoy⁵¹, and Wegner⁵⁵. Note that Barendregt, Burge, Field and Harrison, Hindley and Seldin, and Wegner only name one Church-Rosser theorem.

Bird and Wadler⁶ discuss evaluation models, time and space efficiency and algorithm design techniques.

Brady⁹ and Minsky³⁹ contain details of the halting problem for Turing machines.

Abelson and Sussman¹, Bird and Wadler⁶, Field and Harrison¹⁸, Glaser, Hankin and Till²¹, Henderson³⁰, Henson³¹, Peyton-Jones⁴³ and Revesz⁴⁶ all discuss aspects of lazy evaluation.

Bird and Wadler⁶ provides a thorough account of infinite lists.

Chapter 9 - Functional programming in Standard ML

Wikstrom⁵⁶ provides thorough coverage of SML programming. Harper, MacQueen and Milner²⁵ describes SML informally and gives details of I/O and modules. (The informal description and the I/O details are duplicated in Wikstrom). Harper, Milner and Tofte²⁶ is a first version of a formal semantics for SML.

Bird and Wadler⁶ provide much additional material which is relevant to SML programming including discussion of concrete and abstract types.

Chapter 10 - Functional programming and LISP

Steele³⁴ is the COMMON LISP ‘bible’. Wilensky⁵⁷ is an good introduction. Winston and Horn⁵⁸ take an Artificial Intelligence perspective.

Abelson and Sussman¹ is the standard reference for SCHEME and is based around an introductory computer science course. Dybvig¹⁷ is a more traditional programming language text.

References

1. H. Abelson, G. J. Sussman, and J. Sussman, *Structure and Interpretation of Computer Programs*, MIT Press, Cambridge, Massachusetts, (1985).
2. R. C. Backhouse, *Program Construction and Verification*, Prentice-Hall, Englewood Cliffs, N.J., (1986).
3. J. W. Backus, “Can programming be liberated from the von Neumann style? A functional style and its algebra of programs.” *Communications of the ACM*, **Vol. 21**, (8), pp. 613-641, (August 1978).
4. R. Bailey, “An Introduction to Hope,” in *Functional Programming: Languages, Tools & Architectures*, ed. S. Eisenbach, Ellis Horwood, Chichester, (1987).
5. H. P. Barendregt, *The Lambda Calculus: Its Syntax and Semantics*, North-Holland, Amsterdam, (1981).
6. R. Bird and P. Wadler, *Introduction to Functional Programming*, Prentice-Hall, New York, (1988).
7. R. Bornat, *Programming from First Principles*, Prentice-Hall, Englewood Cliffs N.J., (1987).
8. B. Boutel, “Combinators as Machine Code for Implementing Functional Languages,” in *Functional Programming: Languages, Tools & Architectures*, ed. S. Eisenbach, Ellis Horwood, Chichester, (1987).
9. J. M. Brady, *The Theory of Computer Science: A Programming Approach*, Chapman and Hall, London, (1977).
10. W. H. Burge, *Recursive Programming Techniques*, Addison-Wesley, Reading, (1975).

11. R. M. Burstall, J. S. Collins, and R. J. Poppstone, *Programming in POP-2*, Edinburgh University Press, Edinburgh, (1977).
12. R. Carrick, J. Cole, and R. Morrison, "An Introduction to PS-algol Programming," PPRR 31, Dept. of Computational Science, University of St Andrews, St Andrews, Scotland, (1986).
13. A. Church, *The Calculi of Lambda Conversion*, Princeton University Press, Princeton, N.J., (1941).
14. W. F. Clocksin and C. S. Mellish, *Programming in Prolog*, Springer-Verlag, Berlin, (1981).
15. B. Cohen, W. T. Harwood, and M. I. Jackson, *The Specification of Complex Systems*, Addison-Wesley, Wokingham, (1986).
16. M. Cripps, T. Field, and M. Reeve, "An Introduction to ALICE: a Multiprocessor Graph Reduction Machine," in *Functional Programming: Languages, Tools & Architectures*, ed. S. Eisenbach, Ellis Horwood, Chichester, (1987).
17. R. K. Dybvig, *The SCHEME Programming Language*, Prentice-Hall, Englewood Cliffs, New Jersey, (1987).
18. A. Field and P. Harrison, *Functional Programming*, Addison-Wesley, Wokingham, (1988).
19. W. Findlay and D. A. Watt, *Pascal: An Introduction to Methodical Programming*, Pitman, London, (1981).
20. N. Gehani and A. D. McGettrick, *Software Specification Techniques*, Addison-Wesley, Wokingham, (1986).
21. H. Glaser, C. Hankin, and D. Till, *Principles of Functional Programming*, Prentice-Hall, Englewood-Cliffs, N.J., (1984).
22. M. J. C. Gordon, *The Denotational Description of Programming Languages: An Introduction*, Springer-Verlag, New York, (1979).
23. D. Gries, *The Science of Programming*, Springer-Verlag, New York, (1981).
24. C. Hankin, D. Till, and H. Glaser, "Applicative Languages and Data Flow," in *Functional Programming: Languages, Tools & Architectures*, ed. S. Eisenbach, Ellis Horwood, Chichester, (1987).
25. R. Harper, D. MacQueen, and R. Milner, "Standard ML," ECS-LFCS-86-2, LFCS, Dept. of Computer Science, University of Edinburgh, Edinburgh, Scotland, (March 1986).
26. R. Harper, R. Milner, and M. Tofte, "The Semantics of Standard ML, Version 1," ECS-LFCS-87-36, LFCS, Dept. of Computer Science, University of Edinburgh, Edinburgh, Scotland, (August 1987).
27. P. Harrison and H. Khoshnevisan, "A Functional Algebra and its Application to Program Transformation," in *Functional Programming: Languages, Tools & Architectures*, ed. S. Eisenbach, Ellis Horwood, Chichester, (1987).
28. P. Harrison and H. Khoshnevisan, "An Introduction to FP and the FP Style of Programming," in *Functional Programming: Languages, Tools & Architectures*, ed. S. Eisenbach, Ellis Horwood, Chichester, (1987).
29. I. Hayes, *Specification Case Studies*, Prentice-Hall, Englewood Cliffs, N.J., (1987).
30. P. Henderson, *Functional Programming: Application and Implementation*, Prentice-Hall, Englewood Cliffs, N.J., (1980).
31. M. C. Henson, *Elements of Functional Languages*, Blackwell, Oxford, (1987).

32. J. R. Hindley and J. P. Seldin, *Introduction to Combinators and λ Calculus*, Cambridge University Press, Cambridge, England, (1987).
33. C. B. Jones, *Systematic Software Development Using VDM*, Prentice-Hall, Englewood Cliffs, N.J., (1986).
34. G. L. Steele Jr., *Common LISP: The Language*, Digital, (1984).
35. B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, Prentice-Hall, Englewood Cliffs, N.J., (1978).
36. S. C. Kleene, *Introduction to Metamathematics*, North-Holland, Amsterdam, (1952).
37. Z. Manna, *Mathematical theory of Computation*, McGraw-Hill, New York, (1974).
38. E. Mendelson, *Introduction to Mathematical Logic*, D. Van Nostrand, Princeton, New Jersey, (1964).
39. M. Minsky, *Computation: Finite and Infinite Machines*, Prentice-Hall, London, (1972).
40. P. Naur and et. al., "Revised Report on the Algorithmic Language ALGOL 60," *Communications of the ACM*, **Vol. 6**, (1), (1963).
41. F. G. Pagan, *A Practical Guide to Algol68*, Wiley, London, (1976).
42. R. Peter, *Recursive Functions*, Academic Press, New York, (1967).
43. S. L. Peyton-Jones, *The Implementation of Functional Programming Languages*, Prentice-Hall, Englewood Cliffs, N.J., (1987).
44. C. Queinnec, *LISP*, Macmillan, London, (1983).
45. V. J. Rayward-Smith, *A First Course in Computability*, Blackwell, Oxford, (1986).
46. G. Revesz, *Lambda-Calculus, Combinators and Functional Programming*, Cambridge University Press, Cambridge, (1988).
47. M. Richards, "C. Whitby-Strevens," *BCPL - The Language and its Compiler*, Cambridge University Press, Cambridge, (1982).
48. C. Sadler and S. Eisenbach, "Why Functional Programming?" in *Functional Programming: Languages, Tools & Architectures*, ed. S. Eisenbach, Ellis Horwood, Chichester, (1987).
49. D. A. Schmidt, *Denotational Semantics: A Methodology for Language Development*, Allyn and Bacon, Boston, (1986).
50. S. C. Shapiro, *LISP: An Interactive Approach*, Computer Science Press, Rockville, Maryland, (1986).
51. J. E. Stoy, *Denotational Semantics: The Scott-Strachey Approach to Programming Language Theory*, MIT Press, Cambridge, Massachussets, (1977).
52. D. Turner, *An Introduction to Miranda*. (in Peyton-Jones 1987)
53. D. Turner, *SASL Language Manual*, Dept. of Computational Science, University of St Andrews, St Andrews, Scotland, (December 1976).
54. W. M. Turski and T. S. E. Maibaum, *The Specification of Computer Programs*, Addison-Wesley, Wokingham, (1987).

- 55. P. Wegner, *Programming Languages, Information Structures and Machine Organisation*, McGraw-Hill, London, (1971).
- 56. A. Wikstrom, *Functional Programming Using Standard ML*, Prentice-Hall, London, (1987).
- 57. R. Wilensky, *Common LISPcraft*, Norton, New York, (1986).
- 58. P. H. Winston and B. K. P. Horn, *LISP*, Addison-Wesley, Reading, (1984).

Answers to exercises

Chapter 2

1)

i) $\lambda a.(a \lambda b.(b a))$

```
<function>
<bound variable> - a
<body> - (a  $\lambda b.(b a)$ )
<application>
  <function exp> - <name> - a
  <argument exp> -  $\lambda b.(b a)$ 
    <function>
      <bound variable> - b
      <body> - (b a)
      <application>
        <function exp> - <name> - b
        <argument exp> - <name> - a
```

ii) $\lambda x.\lambda y.\lambda z.((z x) (z y))$

```
<function>
<bound variable> - x
<body>  $\lambda y.\lambda z.((z x) (z y))$ 
  <function>
    <bound variable> - y
    <body> -  $\lambda z.((z x) (z y))$ 
      <function>
        <bound variable> - z
        <body> -  $((z x) (z y))$ 
          <application>
            <function exp> - (z x)
            <application>
              <function exp> - <name> - z
              <argument exp> - <name> - x
            <function exp> - (z y)
            <application>
              <function exp> - <name> - z
              <argument exp> - <name> - y
```

iii) $(\lambda f.\lambda g.(\lambda h.(g h) f) \lambda p.\lambda q.p)$

```
<application>
<function exp>
  <function> -  $\lambda f.\lambda g.(\lambda h.(g h) f)$ 
    <bound variable> - f
    <body> -  $\lambda g.(\lambda h.(g h) f)$ 
      <function>
        <bound variable> - g
        <body> -  $(\lambda h.(g h) f)$ 
          <application>
            <function exp> -  $\lambda h.(g h)$ 
            <function>
              <bound variable> - h
```

```

    <body> - (g h)
    <application>
      <function exp> - <name> - g
      <argument exp> - <name> - h
    <argument exp> - <name> f
  <argument exp> - λp.λq.p
  <function>
    <bound variable> - p
    <body> - λq.p
    <function>
      <bound variable> - q
      <body> - <name> - p

```

iv) λfee.λfi.λfo.λfum.(fum (fo (fi fee)))

```

<function>
  <bound variable> - fee
  <body> - λfi.λfo.λfum.(fum (fo (fi fee)))
  <function>
    <bound variable> - fi
    <body> - λfo.λfum.(fum (fo (fi fee)))
    <function>
      <bound variable> - fo
      <body> - λfum.(fum (fo (fi fee)))
      <function>
        <bound variable> - fum
        <body> - (fum (fo (fi fee)))
        <application>
          <function exp> - <name> - fum
          <argument exp> - (fo (fi fee))
          <application>
            <function exp> - <name> - fo
            <argument exp> - (fi fee)
            <application>
              <function exp> - <name> - fi
              <argument exp> - <name> - fee

```

v) ((λp.(λq.p λx.(x p)) λi.λj.(j i)) λa.λb.(a (a b)))

```

<application>
  <function exp> - (λp.(λq.p λx.(x p)) λi.λj.(j i))
  <application>
    <function exp> - λp.(λq.p λx.(x p))
    <function>
      <bound variable> - p
      <body> - (λq.p λx.(x p))
      <application>
        <function exp> - λq.p
        <function>
          <bound variable> - q
          <body> - <name> - p
        <argument exp> - λx.(x p)
        <function>
          <bound variable> - x
          <body> - (x p)
          <application>
            <function exp> - <name> - x
            <argument exp> - <name> p
        <argument exp> - λi.λj.(j i)

```

```
<function>
  <bound variable> - i
  <body> - λj.(j i)
    <function>
      <bound variable> - j
      <body> - (j i)
        <application>
          <function exp> - <name> - j
          <argument exp> - <name> - i
<argument exp> - λa.λb.(a (a b))
<function>
  <bound variable> - a
  <body> - λb.(a (a b))
    <function>
      <bound variable> - b
      <body>
        <application>
          <function exp> - <name> - a
          <argument exp> - (a b)
            <application>
              <function exp> - <name> - a
              <argument exp> - <name> - b
```

2)

```
i) ((λx.λy.(y x) λp.λq.p) λi.i) =>
  (λy.(y λp.λq.p) λi.i) =>
  (λi.i λp.λq.p) =>
  λp.λq.p

ii) (((λx.λy.λz.((x y) z) λf.λa.(f a)) λi.i) λj.j) =>
  ((λy.λz.((λf.λa.(f a) y) z) λi.i) λj.j) =>
  (λz.((λf.λa.(f a) λi.i) z) λj.j) =>
  ((λf.λa.(f a) λi.i) λj.j) =>
  (λa.(λi.i a) λj.j) =>
  (λi.i λj.j) =>
  λj.j

iii) (λh.((λa.λf.(f a) h) h) λf.(f f)) =>
  ((λa.λf.(f a) λf.(f f)) λf.(f f)) =>
  (λf.(f λf.(f f)) λf.(f f)) =>
  (λf.(f f) λf.(f f)) =>
  (λf.(f f) λf.(f f)) => ...

iv) ((λp.λq.(p q) (λx.x λa.λb.a)) λk.k) =>
  (λq.((λx.x λa.λb.a) q) λk.k) =>
  ((λx.x λa.λb.a) λk.k) =>
  (λa.λb.a λk.k) =>
  λb.λk.k

v) (((λf.λg.λx.(f (g x)) λs.(s s)) λa.λb.b) λx.λy.x) =>
  ((λg.λx.(λs.(s s) (g x)) λa.λb.b) λx.λy.x) =>
  (λx.(λs.(s s) (λa.λb.b x)) λx.λy.x) =>
  (λs.(s s) (λa.λb.b λx.λy.x)) =>
  ((λa.λb.b λx.λy.x) (λa.λb.b λx.λy.x)) =>
  (λb.b (λa.λb.b λx.λy.x)) =>
  (λa.λb.b λx.λy.x) =>
```

$\lambda b.b$

3)

- i) a) $(\text{identity } \langle \text{argument} \rangle) \Rightarrow \dots \Rightarrow \langle \text{argument} \rangle$
b) $((\text{apply } (\text{apply } \text{identity})) \langle \text{argument} \rangle) \Rightarrow \dots \Rightarrow ((\text{apply } \text{identity}) \langle \text{argument} \rangle) \Rightarrow \dots \Rightarrow (\text{identity } \langle \text{argument} \rangle) \Rightarrow \dots \Rightarrow \langle \text{argument} \rangle$
- ii) a) $((\text{apply } \langle \text{function} \rangle) \langle \text{argument} \rangle) \Rightarrow \dots \Rightarrow (\langle \text{function} \rangle \langle \text{argument} \rangle)$
b) $((\lambda x. \lambda y. (((\text{make_pair } x) y) \text{identity}) \langle \text{function} \rangle) \langle \text{argument} \rangle) \Rightarrow (\lambda y. (((\text{make_pair } \langle \text{function} \rangle) y) \text{identity}) \langle \text{argument} \rangle) \Rightarrow (((\text{make_pair } \langle \text{function} \rangle) \langle \text{argument} \rangle) \text{identity}) \Rightarrow \dots \Rightarrow ((\text{identity } \langle \text{function} \rangle) \langle \text{argument} \rangle) \Rightarrow \dots \Rightarrow (\langle \text{function} \rangle \langle \text{argument} \rangle)$
- iii) a) $(\text{identity } \langle \text{argument} \rangle) \Rightarrow \dots \Rightarrow \langle \text{argument} \rangle$
b) $((\text{self_apply } (\text{self_apply } \text{select_second})) \langle \text{argument} \rangle) \Rightarrow \dots \Rightarrow (((\text{self_apply } \text{select_second}) (\text{self_apply } \text{select_second})) \langle \text{argument} \rangle) \Rightarrow \dots \Rightarrow (((\text{select_second } \text{select_second}) (\text{select_second } \text{select_second})) \langle \text{argument} \rangle) \Rightarrow \dots \Rightarrow ((\lambda \text{second}. \text{second } (\text{select_second } \text{select_second}) \langle \text{argument} \rangle) \Rightarrow \dots \Rightarrow ((\text{select_second } \text{select_second}) \langle \text{argument} \rangle) \Rightarrow \dots \Rightarrow (\lambda \text{second}. \text{second } \langle \text{argument} \rangle) \Rightarrow \langle \text{argument} \rangle$

4)

```
def make_triplet =  $\lambda$ first.  
                   $\lambda$ second.  
                   $\lambda$ third.  
                   $\lambda s. (((s \text{ first}) \text{ second}) \text{ third})$   
  
def triplet_first =  $\lambda$ first. $\lambda$ second. $\lambda$ third.first  
  
def triplet_second =  $\lambda$ first. $\lambda$ second. $\lambda$ third.second  
  
def triplet_third =  $\lambda$ first. $\lambda$ second. $\lambda$ third.third  
  
make_triplet  $\langle \text{item1} \rangle$   $\langle \text{item2} \rangle$   $\langle \text{item3} \rangle$  triplet_first ==  
 $\lambda$ first.  
   $\lambda$ second.  
     $\lambda$ third.  
       $\lambda s. (((s \text{ first}) \text{ second}) \text{ third}) \langle \text{item1} \rangle \langle \text{item2} \rangle \langle \text{item3} \rangle$  triplet_first  $\Rightarrow \dots \Rightarrow$   
       $((\text{triplet\_first } \langle \text{item1} \rangle) \langle \text{item2} \rangle) \langle \text{item3} \rangle ==$   
       $((\lambda \text{first}. \lambda \text{second}. \lambda \text{third}. \text{first } \langle \text{item1} \rangle) \langle \text{item2} \rangle) \langle \text{item3} \rangle \Rightarrow \dots \Rightarrow$   
       $\langle \text{item1} \rangle$ 
```

```
make_triplet <item1> <item2> <item3> triplet_first ==
λfirst.
  λsecond.
    λthird.
      λs.(((s first) second) third) <item1> <item2> <item3> triplet_second => ... =>
      (((triplet_second <item1>) <item2>) <item3>) ==
      (((λfirst.λsecond.λthird.second <item1>) <item2>) <item3>) => ... =>
      <item2>

make_triplet <item1> <item2> <item3> triplet_third ==
λfirst.
  λsecond.
    λthird.
      λs.(((s first) second) third) <item1> <item2> <item3> triplet_third => ... =>
      (((triplet_third <item1>) <item2>) <item3>) ==
      (((λfirst.λsecond.λthird.third <item1>) <item2>) <item3>) => ... =>
      <item3>
```

5)

```
i)   λx.λy.(λx.y λy.x)
x bound at {x} in λx.λy.(λx.y λy.{x})
x free at {x} in λy.(λx.y λy.{x})
      (λx.y λy.{x})
      λy.{x}
      {x}
y bound at {y} in λx.λy.(λx.{y} λy.x)
      λy.(λx.{y} λy.x)
y free at {y} in (λx.{y} λy.x)
      λx.{y}
      {y}

ii)  λx.(x (λy.(λx.x y) x))
x bound at {x} in λx.({x} (λy.(λx.x y) {x}))
x free at {x} in ({x} (λy.(λx.x y) {x}))
      in {x}
      in λy.(λx.x y) {x}
      in {x}
x bound at {x} in λy.(λx.{x} y)
      (λx.{x} y)
      λx.{x}
x free at {x} in {x}
y bound at {y} in λx.(x (λy.(λx.x {y}) x))
      (x (λy.(λx.x {y}) x))
      (λy.(λx.x {y}) x)
      λy.(λx.x {y})
y free at {y} in (λx.x {y})
      {y}

iii) λa.(λb.a λb.(λa.a b))

a bound at {a} in λa.(λb.{a} λb.(λa.a b))
a free at {a} in (λb.{a} λb.(λa.a b))
      λb.{a}
      {a}
a bound at {a} in λa.(λb.a λb.(λa.{a} b))
      (λb.a λb.(λa.{a} b))
```

$\lambda b.(\lambda a.\{a\} b))$
 $\lambda a.\{a\}$
a free at {a} in {a}
b bound at {b} in $\lambda a.(\lambda b.a \lambda b.(\lambda a.a \{b\}))$
 $(\lambda b.a \lambda b.(\lambda a.a \{b\}))$
 $\lambda b.(\lambda a.a \{b\})$
b free at {b} in $(\lambda a.a \{b\})$
 $\{b\}$

iv) $(\lambda free.bound \lambda bound.(\lambda free.free bound))$

bound free at {bound} in $(\lambda free.\{bound\} \lambda bound.(\lambda free.free bound))$
 $\lambda free.\{bound\}$
 $\{bound\}$
bound bound at {bound} in $(\lambda free.bound \lambda bound.(\lambda free.free \{bound\}))$
 $\lambda bound.(\lambda free.free \{bound\})$
bound free at {bound} in $(\lambda free.free \{bound\})$
 $\{bound\}$
free bound at {free} in $(\lambda free.bound \lambda bound.(\lambda free.\{free\} bound))$
 $\lambda bound.(\lambda free.\{free\} bound)$
 $(\lambda free.\{free\} bound)$
 $\lambda free.\{free\}$
free free at {free} in {free}

v) $\lambda p.\lambda q.(\lambda r.(p (\lambda q.(\lambda p.(r q)))) (q p))$

p bound at {p} in $\lambda p.\lambda q.(\lambda r.(\{p\} (\lambda q.(\lambda p.(r q)))) (q \{p\}))$
p free at {p} in $\lambda q.(\lambda r.(\{p\} (\lambda q.(\lambda p.(r q)))) (q \{p\}))$
 $(\lambda r.(\{p\} (\lambda q.(\lambda p.(r q)))) (q \{p\}))$
 $\lambda r.(\{p\} (\lambda q.(\lambda p.(r q))))$
 $(\{p\} (\lambda q.(\lambda p.(r q))))$
 $\{p\}$
 $(q \{p\})$
 $\{p\}$
q bound at {q} in $\lambda p.\lambda q.(\lambda r.(p (\lambda q.(\lambda p.(r q)))) (\{q\} p))$
 $\lambda q.(\lambda r.(p (\lambda q.(\lambda p.(r q)))) (\{q\} p))$
q free at {q} in $(\lambda r.(p (\lambda q.(\lambda p.(r q)))) (\{q\} p))$
 $(\{q\} p))$
 $\{q\}$
q bound at {q} in $\lambda p.\lambda q.(\lambda r.(p (\lambda q.(\lambda p.(r \{q\})))) (q p))$
 $\lambda q.(\lambda r.(p (\lambda q.(\lambda p.(r \{q\})))) (q p))$
 $(\lambda r.(p (\lambda q.(\lambda p.(r \{q\})))) (q p))$
 $\lambda r.(p (\lambda q.(\lambda p.(r \{q\}))))$
 $(p (\lambda q.(\lambda p.(r \{q\}))))$
 $\lambda q.(\lambda p.(r \{q\}))))$
q free at {q} in $(\lambda p.(r \{q\}))$
 $(r \{q\})$
 $\{q\}$
r bound at {r} in $\lambda p.\lambda q.(\lambda r.(p (\lambda q.(\lambda p.(\{r\} q)))) (q p))$
 $\lambda q.(\lambda r.(p (\lambda q.(\lambda p.(\{r\} q)))) (q p))$
 $(\lambda r.(p (\lambda q.(\lambda p.(\{r\} q)))) (q p))$
 $\lambda r.(p (\lambda q.(\lambda p.(\{r\} q))))$
r free at {r} in $(p (\lambda q.(\lambda p.(\{r\} q))))$
 $\lambda q.(\lambda p.(\{r\} q))$
 $(\lambda p.(\{r\} q))$
 $(\{r\} q)$
 $\{r\}$

6)

- i) $\lambda x. \lambda y. (\lambda z. y \ \lambda a. x)$
- ii) $\lambda x. (x \ (\lambda y. (\lambda z. z \ y) \ x))$
- iii) $\lambda a. (\lambda b. a \ \lambda b. (\lambda c. c \ b))$
- v) $\lambda p. \lambda q. (\lambda r. (p \ (\lambda q. (\lambda s. (r \ q)))) \ (q \ p))$

Chapter 3

1)

```
def implies =  $\lambda x. \lambda y. (x \ y \ \text{true})$   
  
implies false false => ... => false false true => ... => true  
implies false true => ... => false true true => ... => true  
implies true false => ... => true false true => ... => false  
implies true true => ... => true true true => ... => true
```

2)

```
def equiv =  $\lambda x. \lambda y. (x \ y \ (\text{not } y))$   
  
equiv false false => ... => false false (not false) => ... => true  
equiv false true => ... => false true (not true) => ... => false  
equiv true false => ... => true false (not false) => ... => false  
equiv true true => ... => true true (not true) => ... => true
```

3)

- i) a) $\lambda x. \lambda y. (\text{and } (\text{not } x) \ (\text{not } y))$ false false => ... =>
and (not false) (not false) => ... =>
(not false) (not false) false => ... =>
true (not false) false => ... =>
not false => ... => true

 $\lambda x. \lambda y. (\text{and } (\text{not } x) \ (\text{not } y))$ false true => ... =>
and (not false) (not true) => ... =>
(not false) (not true) false => ... =>
true (not true) false => ... =>
not true => ... => false

 $\lambda x. \lambda y. (\text{and } (\text{not } x) \ (\text{not } y))$ true false => ... =>
and (not true) (not false) => ... =>
(not true) (not false) false => ... =>
false (not false) false => .. => false

 $\lambda x. \lambda y. (\text{and } (\text{not } x) \ (\text{not } y))$ true true => ... =>
and (not true) (not true)
(not true) (not true) false => ... =>
false (not true) false => ... => false

b) $\lambda x. \lambda y. (\text{not } (\text{or } x \ y))$ false false => ... =>

```
not (or false false) => ... =>
(or false false) false true => ... =>
(false true false) false true => ... =>
false false true => ... => true
```

```
λx.λy.(not (or x y)) false true => ... =>
not (or false true) => ... =>
(or false true) false true => ... =>
(false true true) false true => ... =>
true false true => ... => false
```

```
λx.λy.(not (or x y)) true false => ... =>
not (or true false) => ... =>
(or true false) false true => ... =>
(true true false) false true => ... =>
true false true => ... => false
```

```
λx.λy.(not (or x y)) true true => ... =>
not (or true true) => ... =>
(or true true) false true => ... =>
(true true true) false true => ... =>
true false true => ... => false
```

ii) a) - see 1) above

b) $\lambda x.\lambda y.(\text{implies } (\text{not } y) (\text{not } x)) \text{ false false} \Rightarrow \dots \Rightarrow$
 $\text{implies } (\text{not false}) (\text{not false}) \Rightarrow \dots \Rightarrow$
 $(\text{not false}) (\text{not false}) \text{ true} \Rightarrow \dots \Rightarrow$
 $\text{true } (\text{not false}) \text{ true} \Rightarrow \dots \Rightarrow$
 $\text{not false} \Rightarrow \dots \Rightarrow \text{true}$

```
λx.λy.(implies (not y) (not x)) false true => ... =>
implies (not true) (not false) => ... =>
(not true) (not false) true => ... =>
false (not false) true => ... => true
```

```
λx.λy.(implies (not y) (not x)) true false => ... =>
implies (not false) (not true)
(not false) (not true) true => ... =>
true (not true) true => ... =>
not true => ... => false
```

```
λx.λy.(implies (not y) (not x)) true true => ... =>
implies (not true) (not true) => ... =>
(not true) (not true) true => ... =>
false (not true) true => ... => true
```

iii) a) $\text{not false} \Rightarrow \dots \Rightarrow \text{true}$
 $\text{not true} \Rightarrow \dots \Rightarrow \text{false}$

b) $\lambda x.(\text{not } (\text{not } (\text{not } x))) \text{ false} \Rightarrow \dots \Rightarrow$
 $\text{not } (\text{not } (\text{not false})) \Rightarrow \dots \Rightarrow$
 $(\text{not } (\text{not false})) \text{ false true} \Rightarrow \dots \Rightarrow$
 $((\text{not false}) \text{ false true}) \text{ false true} \Rightarrow \dots \Rightarrow$
 $((\text{false false true}) \text{ false true}) \text{ false true} \Rightarrow \dots \Rightarrow$
 $(\text{true false true}) \text{ false true} \Rightarrow \dots \Rightarrow$
 $\text{false false true} \Rightarrow \dots \Rightarrow \text{true}$

```
λx.(not (not (not x))) true => ... =>
not (not (not true)) => ... =>
((not (not true)) false true) => ... =>
((not true) false true) false true => ... =>
((true false true) false true) false true => ... =>
(false false true) false true => ... =>
true false true => ... => false
```

iv) a) - see 1) above

b) $\lambda x.\lambda y.(\text{not } (\text{and } x (\text{not } y)))$ false false => ... =>
not (and false (not false)) => ... =>
(and false (not false)) false true => ... =>
(false (not false) false) false true => ... =>
false false true => ... => true

$\lambda x.\lambda y.(\text{not } (\text{and } x (\text{not } y)))$ false true => ... =>
not (and false (not true)) => ... =>
(and false (not true)) false true => ... =>
(false (not true) false) false true => ... =>
false false true => ... => true

$\lambda x.\lambda y.(\text{not } (\text{and } x (\text{not } y)))$ true false => ... =>
not (and true (not false)) => ... =>
(and true (not false)) false true => ... =>
(true (not false) false) false true => ... =>
(not false) false true => ... =>
true false true => ... => false

$\lambda x.\lambda y.(\text{not } (\text{and } x (\text{not } y)))$ true true => ... =>
not (and true (not true)) => ... =>
(and true (not true)) false true => ... =>
(true (not true) false) false true => ... =>
(not true) false true => ... =>
false false true => ... => true

v) a) - see 2) above

b) $\lambda x.\lambda y.(\text{and } (\text{implies } x y) (\text{implies } y x))$ false false => ... =>
and (implies false false) (implies false false) => ... =>
(implies false false) (implies false false) false => ... =>
(false false true) (implies false false) false => ... =>
true (implies false false) false => ... =>
implies false false => ... =>
false false true => ... => true

$\lambda x.\lambda y.(\text{and } (\text{implies } x y) (\text{implies } y x))$ false true => ... =>
and (implies false true) (implies true false)
(implies false true) (implies true false) false => ... =>
(false true true) (implies true false) false => ... =>
true (implies true false) false => ... =>
implies true false => ... =>
true false false => ... => false

$\lambda x.\lambda y.(\text{and } (\text{implies } x y) (\text{implies } y x))$ true false => ... =>
and (implies true false) (implies false true)
(implies true false) (implies false true) false => ... =>
(true false true) (implies false true) false => ... =>

```
false (implies false true) false => ... => false

λx.λy.(and (implies x y) (implies y x)) true true => ... =>
and (implies true true) (implies true true) => ... =>
(implies true true) (implies true true) false => ... =>
(true true true) (implies true true) false => ... =>
true (implies true true) false => ... =>
implies true true => ... =>
true true true => ... => true
```

4)

```
λx.(succ (pred x)) λs.(s false <number>) =>
succ (pred λs.(s false <number>))
Simplifying: pred λs.(s false <number>) => ... =>
<number>
so: succ <number> => ... =>
λs.(s false <number>)

λx.(pred (succ x)) λs.(s false <number>) =>
pred (succ λs.(s false <number>))
Simplifying: succ λs.(s false <number>) => ... =>
λs.(s false λs.(s false <number>))
so: pred λs.(s false λs.(s false <number>)) => ... =>
λs.(s false <number>)

λx.(succ (pred x)) zero =>
(succ (pred zero))
Simplifying: pred zero => ... => zero
so: succ zero ==
one

λx.(pred (succ x)) zero =>
(pred (succ zero))
Simplifying: succ zero == one
so: pred one => ... =>
zero
```

Chapter 4

1)

```
sum three => ... =>
recursive sum1 three => ... =>
sum1 (recursive sum1) three => ... =>
add three ((recursive sum1) (pred three)) -> ... ->
add three (sum1 (recursive sum1) two) -> ... ->
add three (add two ((recursive sum1) (pred two))) -> ... ->
add three (add two (sum1 (recursive sum1) one)) -> ... ->
add three (add two (add one ((recursive sum1) (pred one)))) -> ... ->
add three (add two (add one (sum1 (recursive sum1) zero))) -> ... ->
add three (add two (add one zero)) -> ... ->
six
```

2)

```
def prod1 f n =
  if equal n one
  then one
  else mult n (f (pred n))

def prod = recursive prod1

prod three
recursive prod1 three => ... =>
prod1 (recursive prod1) three => ... =>
mult three ((recursive prod1) (pred three)) -> ... ->
mult three (prod1 (recursive prod1) two) -> ... ->
mult three (mult two ((recursive prod1) (pred two))) -> ... ->
mult three (mult two (prod1 (recursive prod1) one)) -> ... ->
mult three (mult two one) -> ... ->
six
```

3)

```
def fun_sum1 f fun n =
  if iszero n
  then fun zero
  else add (fun n) (f fun (pred n))

def fun_sum = recursive fun_sum1

fun_sum double three => ... =>
recursive fun_sum1 double three => ... =>
fun_sum1 (recursive fun_sum1) double three => ... =>
add (double three)
  ((recursive fun_sum1) double (pred three)) -> ... ->
add (double three)
  (fun_sum1 (recursive fun_sum1) double two) -> ... ->
add (double three)
  (add (double two)
    ((recursive fun_sum1) double (pred two))) -> ... ->
add (double three)
  (add (double two)
    (fun_sum1 (recursive fun_sum1) double one)) -> ... ->
add (double three)
  (add (double two)
    (add (double one)
      ((recursive fun_sum1) double (pred one)))) -> ... ->
add (double three)
  (add (double two)
    (add (double one)
      (fun_sum1 (recursive fun_sum1) double zero))) -> ... ->
add (double three)
  (add (double two)
    (add (double one)
      (double zero))) -> ... ->
twelve
```

4)

```
def fun_sum_step1 f fun n s =
  if iszero n
```

```
then fun n
else add (fun n) (f fun (sub n s) s)
```

```
def fun_sum_step = recursive fun_sum_step1
```

```
i)
fun_sum_step double five two => ... =>
recursive fun_sum_step1 double five two => ... =>
fun_sum_step1 (recursive fun_sum_step1) double five two => ... =>
add (double five)
  ((recursive fun_sum_step1) double (sub five two) two) -> ... ->
add (double five)
  (fun_sum_step1 (recursive fun_sum_step1) double three two) -> ... ->
add (double five)
  (add (double three)
    ((recursive fun_sum_step1) double (sub three two) two)) -> ... ->
add (double five)
  (add (double three)
    (fun_sum_step1 (recursive fun_sum_step1) double one two)) -> ... ->
add (double five)
  (add (double three)
    (add (double one)
      ((recursive fun_sum_step1) double (sub one two) two))) -> ... ->
add (double five)
  (add (double three)
    (add (double one)
      (fun_sum_step1 (recursive fun_sum_step1) double zero two))) -> ... ->
add (double five)
  (add (double three)
    (add (double one)
      (double zero))) -> ... ->
eighteen
```

```
ii)
fun_sum_step double four two
recursive fun_sum_step1 double four two => ... =>
fun_sum_step1 (recursive fun_sum_step1) double four two => ... =>
add (double four)
  ((recursive fun_sum_step1) double (sub four two) two) -> ... ->
add (double four)
  (fun_sum_step1 (recursive fun_sum_step1) double two two) -> ... ->
add (double four)
  (add (double three)
    ((recursive fun_sum_step1) double (sub two two) two)) -> ... ->
add (double four)
  (add (double two)
    (fun_sum_step1 (recursive fun_sum_step1) double zero two)) -> ... ->
add (double four)
  (add (double two)
    (double zero))
twelve
```

5)

```
def less x y = greater y x
```

```
def less_or_equal x y = greater_or_equal y x
```

- i) less three two => ... =>
 greater two three => ... =>
 not (iszero (sub two three)) -> ... ->
 not (iszero zero) -> ... ->
 not true => ... => false
- ii) less two three => ... =>
 greater three two -> ... -> true - see 4.8.3
- iii) less two two => ... =>
 greater two two => ... =>
 not (iszero (sub two two)) -> ... ->
 not (iszero zero) -> ... ->
 not true => ... => false
- iv) less_or_equal three two => ... =>
 greater_or_equal two three => ... =>
 iszero (sub three two) -> ... ->
 iszero one => ... => false
- v) less_or_equal two three => ... =>
 greater_or_equal three two => ... =>
 iszero (sub two three) -> ... ->
 iszero zero => ... => true
- vi) less_or_equal two two => ... =>
 greater_or_equal two two => ... =>
 iszero (sub two two) -> ... ->
 iszero zero => ... => true

6)

```
def mod x y =  
  if iszero y  
  then x  
  else mod1 x y
```

```
rec mod1 x y =  
  if less x y  
  then x  
  else mod1 (sub x y) y
```

- i) mod three two => ... =>
 mod1 three two
 mod1 (sub three two) two -> ... ->
 mod1 one two => ... => one
- ii) mod two three => ... =>
 mod1 two three => ... => two
- iii) mod three zero => ... => three

Chapter 5

1)

```
i)  ISBOOL 3 => ... =>
    MAKE_BOOL (isbool 3) ==
    MAKE_BOOL (istype bool_type 3) -> ... ->
    MAKE_BOOL (equal (type 3) bool_type) -> ... ->
    MAKE_BOOL (equal numb_type bool_type) -> ... ->
    MAKE_BOOL false ==
    FALSE

ii) ISNUMB FALSE => ... =>
    MAKE_BOOL (isnumb FALSE) ==
    MAKE_BOOL (istype numb_type FALSE) -> ... ->
    MAKE_BOOL (equal (type FALSE) numb_type) -> ... ->
    MAKE_BOOL (equal bool_type numb_type) -> ... ->
    MAKE_BOOL false ==
    FALSE

iii) NOT 1 => ... =>
    if isbool 1
    then MAKE_BOOL (not (value 1))
    else BOOL_ERROR -> ... ->
    if equal (type 1) bool_type
    then ...
    else ... -> ... ->
    if equal numb_type bool_type
    then ...
    else ... -> ... ->
    if false
    then ...
    else BOOL_ERROR -> ... ->
    BOOL_ERROR

iv) TRUE AND 2 => ... =>
    if and (isbool TRUE) (isbool 2)
    then MAKE_BOOL (and (value TRUE) (value 2))
    else BOOL_ERROR -> ... ->
    if and (istype bool_type TRUE) (istype bool_type 2)
    then ...
    else ... -> ... ->
    if and (equal (type TRUE) bool_type) (equal (type 2) bool_type)
    then ...
    else ... -> ... ->
    if and (equal bool_type bool_type) (equal numb_type bool_type)
    then ...
    else ... -> ... ->
    if and true false
    then ...
    else ... -> ... ->
    if false
    then ...
    else BOOL_ERROR -> ... ->
    BOOL_ERROR

v)  2 + TRUE => ... =>
    if and (isnumb 2) (isnumb TRUE)
    then MAKE_NUMB (add (value 2) (value TRUE))
    else NUMB_ERROR -> ... ->
    if and (istype numb_type 2) (istype numdtype TRUE)
    then ...
```

```
else ... -> ... ->
if and (equal (type 2) numb_type) (equal (type TRUE) numb_type)
then ...
else ... -> ... ->
if and (equal numb_type numb_type) (equal numb_type numb_type)
then ...
else ... -> ... ->
if and true false
then ...
else NUMB_ERROR -> ... ->
NUMB_ERROR
```

2) i)

```
def issigned N = istype signed_type N

def ISSIGNED N = MAKE_BOOL (issigned N)

def sign = value (select_first (value N))

def SIGN N =
  if issigned N
  then select_first (value N)
  else SIGN_ERROR

def sign_value N = value (select_second (value N))

def VALUE N =
  if issigned N
  then select_second (value N)
  else SIGN_ERROR

def sign_iszero N = iszero (sign_value N)
```

ii)

```
def SIGN_ISZERO N =
  if issigned N
  then MAKE_BOOL (sign_iszero N)
  else SIGN_ERROR

def SIGN_SUCC N =
  IF SIGN_ISZERO N
  THEN +1
  ELSE
    IF SIGN N
    THEN MAKE_SIGNED POS (MAKE_NUMB (succ (sign_value N)))
    ELSE MAKE_SIGNED NEG (MAKE_NUMB (pred (sign_value N)))

def SIGN_PRED N =
  IF SIGN_ISZERO N
  THEN -1
  ELSE
    IF SIGN N
    THEN MAKE_SIGNED POS (MAKE_NUMB (pred (sign_value N)))
    ELSE MAKE_SIGNED NEG (MAKE_NUMB (succ (sign_value N)))
```

iii)

```
def SIGN_+ X Y =
  if and (issigned X) (issigned Y)
  then
    if iszero (sign_value X)
    then Y
    else
      if sign_iszero (sign_value Y)
      then X
      else
        if and (sign X) (sign Y)
        then MAKE_SIGNED POS (MAKE_NUMB (add (sign_value X) (sign_value Y)))
        else
          if and (not (sign X)) (not (sign Y))
          then MAKE_SIGNED NEG (MAKE_NUMB (add (sign_value X) (sign_value Y)))
          else
            if not (sign X)
            then
              if greater (sign_value X) (sign_value Y)
              then MAKE_SIGNED NEG (MAKE_NUMB (sub (sign_value X) (sign_value Y)))
              else MAKE_SIGNED POS (MAKE_NUMB (sub (sign_value Y) (sign_value X)))
            else
              if GREATER (sign_value Y) (sign_value X)
              then MAKE_SIGNED NEG (sub (sign_value Y) (sign_value X))
              else MAKE_SIGNED POS (sub (sign_value X) (sign_value Y))
          else SIGN_ERROR
```

Chapter 6

1)

```
def ATOMCONS A L =
  if isnil L
  then [A]
  else
    if equal (type A) (type (HEAD L))
    then CONS A L
    else LIST_ERROR
```

2)

```
i) rec STARTS [] L = TRUE
   or STARTS L [] = FALSE
   or STARTS (H1::T1) (H2::T2) =
     IF CHAR_EQUALS H1 H2
     THEN STARTS T1 T2
     ELSE FALSE

ii) rec CONTAINS L [] = FALSE
    or CONTAINS L1 L2 =
      IF STARTS L1 L2
      THEN TRUE
      ELSE CONTAINS L1 (TAIL L2)

iii) rec COUNT L [] = 0
      or COUNT L1 L2 =
```

```
IF STARTS L1 L2
THEN 1 + (COUNT L1 (TAIL L2))
ELSE COUNT L1 (TAIL L2)
```

```
iv) rec REMOVE [] L = L
    or REMOVE (H1::T1) (H2::T2) = REMOVE T1 T2
```

```
v) rec DELETE L [] = []
    or DELETE L1 L2 =
      IF STARTS L1 L2
      THEN REMOVE L1 L2
      ELSE (HEAD L2)::(DELETE L1 (TAIL L2))
```

```
vi) rec INSERT L1 L2 [] = []
    or INSERT L1 L2 L3 =
      IF STARTS L2 L3
      THEN APPEND L2 (APPEND L1 (REMOVE L2 L3))
      ELSE (HEAD L3)::(INSERT L1 L2 (TAIL L3))
```

```
vii) rec REPLACE L1 L2 [] = []
    or REPLACE L1 L2 L3 =
      IF STARTS L2 L3
      THEN APPEND L1 (REMOVE L2 L3)
      ELSE (HEAD L3)::(REPLACE L1 L2 (TAIL L3))
```

3)

```
i) rec MERGE L [] = L
    or MERGE [] L = L
    or MERGE (H1::T1) (H2::T2) =
      IF LESS H1 H2
      THEN H1::(MERGE T1 (H2::T2))
      ELSE H2::(MERGE (H1::T1) T2)
```

```
ii) rec LMERGE [] = []
    or LMERGE (H::T) = MERGE H (LMERGE T)
```

Chapter 7

1)

```
i) def T00_SECS [H,M,S] = (60 * ((60 * H) + M)) + S
```

```
def MOD X Y = X - ((X / Y) * Y)
```

```
def FROM_SECS S =
  let SECS = MOD S 60
  in
    let MINS = (MOD S 3600) / 60
    in
      let HOURS = S / 3600
      in [HOURS,MINS,SECS]
```

```
ii) def TICK [H,M,S] =
  let S = S + 1
  in
    IF LESS S 60
```

```
THEN [H,M,S]
ELSE
  let M = M + 1
  in
    IF LESS M 60
    THEN [H,M,0]
    ELSE
      let H = H + 1
      in
        IF LESS H 24
        THEN [H,0,0]
        ELSE [0,0,0]
```

```
iii) def TLESS [TR1,[H1,M1,S1]] [TR2,[H2,M2,S2]] =
  IF LESS H1 H2
  THEN TRUE
  ELSE
    IF EQUAL H1 H2
    THEN
      IF LESS M1 M2
      THEN TRUE
      ELSE
        IF EQUAL M1 M2
        THEN
          IF LESS S1 S2
          THEN TRUE
          ELSE FALSE
        ELSE FALSE
      ELSE FALSE
```

```
rec TINSERT T [] = [T]
or TINSERT T (T1::R) =
  IF TLESS T T1
  THEN T::T1::R
  ELSE T1::(TINSERT T R)
```

```
rec TSORT [] = []
or TSORT (H::T) = TINSERT H (TSORT T)
```

2)

```
i) rec TCOMP EMPTY EMPTY = TRUE
or TCOMP EMPTY T = FALSE
or TCOMP T EMPTY = FALSE
or TCOMP [V1,L1,R1] [V2,L2,R2] =
  IF EQUAL V1 V2
  THEN AND (TCOMP L1 L2) (TCOMP R1 R2)
  ELSE FALSE
```

```
ii) rec TFIND EMPTY T = TRUE
or TFIND T EMPTY = FALSE
or TFIND T1 T2 =
  IF TCOMP T1 T2
  THEN TRUE
  ELSE
    IF LESS (ITEM T1) (ITEM T2)
    THEN TFIND T1 (LEFT T2)
    ELSE TFIND T1 (RIGHT T2)
```

```
iii) rec DTRAVERSE TEMPTY = []  
      or DTRAVERSE [V,L,R] = APPEND (DTRAVERSE R) (V::(DTRAVERSE L))
```

3)

```
rec EVAL [E1,OP,E2] =  
  let R1 = EVAL E1  
  in  
    let R2 = EVAL E2  
    in  
      IF STRING_EQUAL OP "+"  
      THEN R1 + R2  
      ELSE  
        IF STRING_EQUAL OP "-"  
        THEN R1 - R2  
        ELSE  
          IF STRING_EQUAL OP "*"  
          THEN R1 * R2  
          ELSE R1 / R2  
    or EVAL N = N
```

Chapter 8

1)

```
i)  
Normal order  
λs.(s s) (λf.λa.(f a) λx.x λy.y) =>  
(λf.λa.(f a) λx.x λy.y) (λf.λa.(f a) λx.x λy.y) =>  
(λa.(λx.x a) λy.y) (λf.λa.(f a) λx.x λy.y) =>  
(λx.x λy.y) (λf.λa.(f a) λx.x λy.y) =>  
λy.y (λf.λa.(f a) λx.x λy.y) =>  
λf.λa.(f a) λx.x λy.y =>  
λa.(λx.x a) λy.y =>  
λx.x λy.y =>  
λy.y
```

8 reductions

λf.λa.(f a) λx.x λy.y reduced twice

Applicative order

```
λs.(s s) (λf.λa.(f a) λx.x λy.y) ->  
λs.(s s) (λa.(λx.x a) λy.y) ->  
λs.(s s) (λx.x λy.y) ->  
λs.(s s) λy.y ->  
λy.y λy.y ->  
λy.y
```

5 reductions

λf.λa.(f a) λx.x λy.y reduced once

Lazy

```
λs.(s s) (λf.λa.(f a) λx.x λy.y)1 =>  
(λf.λa.(f a) λx.x λy.y)1 (λf.λa.(f a) λx.x λy.y)1 =>  
(λa.(λx.x a) λy.y)2 (λa.(λx.x a) λy.y)2 =>  
(λx.x λy.y)3 (λx.x λy.y)3 =>  
λy.y λy.y =>
```

$\lambda y.y$

5 reductions

$\lambda f.\lambda a.(f\ a)\ \lambda x.x\ \lambda y.y$ reduced once

ii)

Normal order

$\lambda x.\lambda y.x\ \lambda x.x\ (\lambda s.(s\ s)\ \lambda s.(s\ s)) \Rightarrow$

$\lambda y.\lambda x.x\ (\lambda s.(s\ s)\ \lambda s.(s\ s)) \Rightarrow$

$\lambda x.x$

2 reductions

$\lambda s.(s\ s)\ \lambda s.(s\ s)$ not reduced

Applicative order

$\lambda x.\lambda y.x\ \lambda x.x\ (\lambda s.(s\ s)\ \lambda s.(s\ s)) \rightarrow$

$\lambda x.\lambda y.x\ \lambda x.x\ (\lambda s.(s\ s)\ \lambda s.(s\ s)) \rightarrow \dots$

Non-terminating - 1 reduction/cycle

$\lambda s.(s\ s)\ \lambda s.(s\ s)$ reduced every cycle

Lazy

$\lambda x.\lambda y.x\ \lambda x.x\ (\lambda s.(s\ s)\ \lambda s.(s\ s)) \Rightarrow$

$\lambda y.\lambda x.x\ (\lambda s.(s\ s)\ \lambda s.(s\ s)) \Rightarrow$

$\lambda x.x$

2 reductions - as normal order

iii)

Normal order

$\lambda a.(a\ a)\ (\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x) \Rightarrow$

$(\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x)\ (\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x) \Rightarrow$

$\lambda s.(\lambda x.x\ (s\ s))\ (\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x) \Rightarrow$

$\lambda x.x\ ((\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x)\ (\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x)) \Rightarrow$

$(\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x)\ (\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x) \Rightarrow \dots$

Non-terminating - 3 reductions/cycle

$\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x$ reduced every cycle

Applicative order

$\lambda a.(a\ a)\ (\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x) \rightarrow$

$\lambda a.(a\ a)\ \lambda s.(\lambda x.x\ (s\ s)) \rightarrow$

$\lambda s.(\lambda x.x\ (s\ s))\ \lambda s.(\lambda x.x\ (s\ s)) \rightarrow$

$\lambda x.x\ (\lambda s.(\lambda x.x\ (s\ s))\ \lambda s.(\lambda x.x\ (s\ s))) \rightarrow$

$\lambda s.(\lambda x.x\ (s\ s))\ \lambda s.(\lambda x.x\ (s\ s)) \rightarrow \dots$

Non-terminating - 2 reductions/cycle

$\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x$ reduced before non-terminating cycle

Lazy

$\lambda a.(a\ a)\ (\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x)_1 \Rightarrow$

$(\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x)_1\ (\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x)_1 \Rightarrow$

$\lambda s.(\lambda x.x\ (s\ s))\ \lambda s.(\lambda x.x\ (s\ s)) \Rightarrow$

$\lambda x.x\ (\lambda s.(\lambda x.x\ (s\ s))\ \lambda s.(\lambda x.x\ (s\ s))) \Rightarrow$

$\lambda s.(\lambda x.x\ (s\ s))\ \lambda s.(\lambda x.x\ (s\ s)) \Rightarrow \dots$

Non-terminating - 2 reductions/cycle

$\lambda f.\lambda s.(f\ (s\ s))\ \lambda x.x$ reduced before non-terminating cycle

Chapter 9

1)

```
i) fun cube (y:int) = y*y*y;

ii) fun implies (x:bool) (y:bool) = (not x) orelse y;

iii) fun smallest (a:int) (b:int) (c:int) =
      if a<b
      then
        if a<c
        then a
        else c
      else
        if b<c
        then b
        else c;

iv) fun desc_join (s1:string) (s2:string) =
      if s1<s2
      then s1^s2
      else s2^s1;

v) fun shorter (s1:string) (s2:string) =
      if (size s1) < (size s2)
      then s1
      else s2;
```

2)

```
i) fun sum 0 = 0 |
      sum (n:int) = n+(sum (n-1));

ii) fun nsum (m:int) (n:int) =
      if m>n
      then 0
      else m+(nsum (m+1) n);

iii) fun repeat (s:string) 0 = "" |
       repeat (s:string) (n:int) = s^(repeat s (n-1));
```

3)

```
i) fun ncount [] = 0 |
      ncount ((h::t):int list) =
        if h < 0
        then 1+(ncount t)
        else ncount t;

ii) fun scout (s:string) [] = 0 |
       scout (s:string) ((h::t):string list) =
        if h = s
        then 1+(scout s t)
        else scout s t;

iii) fun gconstr (v:int) [] = [] |
```

```
gconstr (v:int) ((h::t):int list) =
  if h > v
  then h::(gconstr v t)
  else gconstr v t;

iv) fun smerge [] (s2:string list) = s2 |
     smerge (s1:string list) [] = s1 |
     smerge ((h1::t1):string list) ((h2::t2):string list) =
       if h1 < h2
       then h1::(smerge t1 (h2::t2))
       else h2::(smerge (h1::t1) t2);

v) fun slmerge [] = [] |
     slmerge ((h::t):(string list) list) = smerge h (slmerge t);

vi)
a) type stock = string * int * int;
   fun item (s:string,n:int,r:int) = s;
   fun numb (s:string,n:int,r:int) = n;
   fun reord (s:string,n:int,r:int) = r;

   fun getmore [] = [] |
       getmore ((h::t):stock list) =
         if (numb h) < (reord h)
         then h::(getmore t)
         else getmore t;

b) type upd = string * int;
   fun uitem (s:string,n:int) = s;
   fun unumb (s:string,n:int) = n;

   fun update1 [] (u:upd) = [] |
       update1 ((h::t):stock list) (u:upd) =
         if (item h) = (uitem u)
         then (item h,(numb h)+(unumb u),reord h)::t
         else h::(update1 t u);

   fun update (r:stock list) [] = r |
       update (r:stock list) ((h::t):upd list) = update (update1 r h) t;

4)

i) fun left1 0 (s:string list) = "" |
     left1 (n:int) [] = "" |
     left1 (n:int) ((h::t):string list) = h^(left1 (n-1) t);

   fun left (n:int) (s:string) = left1 n (explode s);

ii) fun drop 0 (s:string list) = s |
     drop (n:int) [] = [] |
     drop (n:int) ((h::t):string list) = drop (n-1) t;

   fun right (n:int) (s:string) = implode (drop ((size s)-n) (explode s));

iii) fun middle (n:int) (l:int) (s:string) = left1 l (drop (n-1) (explode s));

iv) fun starts [] (s2:string list) = true |
     starts (s1:string list) [] = false |
```

```
starts ((h1::t1):string list) ((h2::t2):string list) =
  if h1=h2
  then starts t1 t2
  else false;

fun find1 [] (s2:string list) = 1 |
  find1 (s1:string list) [] = 1 |
  find1 (s1:string list) (s2:string list) =
    if starts s1 s2
    then 1
    else 1+(find1 s1 (tl s2));

fun find (s1:string) (s2:string) =
  let val pos = find1 (explode s1) (explode s2)
  in
    if pos > (size s2)
    then 0
    else pos
  end;
```

5)

```
fun east Queens_Street = Bishopbriggs |
  east Bishopbriggs = Lenzie |
  east Lenzie = Croy |
  east Croy = Polmont |
  east Polmont = Falkirk_High |
  east Falkirk_High = Linlithgow |
  east Linlithgow = Haymarket |
  east Haymarket = Waverly |
  east Waverly = Waverly;

fun west Queens_Street = Queens_Street |
  west Bishopbriggs = Queens_Street |
  west Lenzie = Bishopbriggs |
  west Croy = Lenzie |
  west Polmont = Croy |
  west Falkirk_High = Polmont |
  west Linlithgow = Falkirk_High |
  west Haymarket = Linlithgow |
  west Waverly = Haymarket;
```

6)

```
fun eval (numb(i:int)) = i |
  eval (add(e1:exp,e2:exp)) = (eval e1)+(eval e2) |
  eval (diff(e1:exp,e2:exp)) = (eval e1)-(eval e2) |
  eval (mult(e1:exp,e2:exp)) = (eval e1)*(eval e2) |
  eval (quot(e1:exp,e2:exp)) = (eval e1) div (eval e2);
```

Chapter 10

1)

```
i) (defun nsum (n)
  (if (eq 0 n)
```

```
0
(+ n (nsum (- n 1))))))
```

```
ii) (defun nprod (n)
      (if (eq 1 n)
          1
          (* n (nprod (- n 1)))))
```

```
iii) (defun napply (fun n)
        (if (eq 0 n)
            (funcall fun 0)
            (+ (funcall fun n) (napply fun (- n 1)))))
```

```
iv) (defun nstepapply (fun n s)
      (if (<= n 0)
          (funcall fun 0)
          (+ (funcall fun n) (nstepapply fun (- n s) s))))
```

2)

```
i) (defun lstarts (l1 l2)
     (cond ((null l1) t)
           ((null l2) nil)
           ((eq (car l1) (car l2)) (lstarts (cdr l1) (cdr l2)))
           (t nil)))
```

```
ii) (defun lcontains (l1 l2)
      (cond ((null l2) nil)
            ((lstarts l1 l2) t)
            (t (lcontains l1 (cdr l2)))))
```

```
iii) (defun lcount (l1 l2)
        (cond ((null l2) 0)
              ((lstarts l1 l2) (+ 1 (lcount l1 (cdr l2))))
              (t (lcount l1 (cdr l2)))))
```

```
iv) (defun lremove (l1 l2)
      (if (null l1)
          l2
          (lremove (cdr l1) (cdr l2))))
```

```
v) (defun ldelete (l1 l2)
     (cond ((null l2) nil)
           ((lstarts l1 l2) (lremove l1 l2))
           (t (cons (car l2) (ldelete l1 (cdr l2)))))
```

```
vi) (defun linsert (l1 l2 l3)
      (cond ((null l3) nil)
            ((lstarts l2 l3) (append l2 (append l1 (lremove l2 l3))))
            (t (cons (car l3) (linsert l1 l2 (cdr l3)))))
```

```
vii) (defun lreplace (l1 l2 l3)
        (cond ((null l3) nil)
              ((lstarts l1 l3) (append l2 (lremove l1 l3)))
              (t (cons (car l3) (lreplace l1 l2 (cdr l3)))))
```

3)

```
i) (defun merge (l1 l2)
    (cond ((null l1) l2)
          ((null l2) l1)
          ((< (car l1) (car l2)) (cons (car l1) (merge (cdr l1) l2)))
          (t (cons (car l2) (merge l1 (cdr l2))))))
```

```
ii) (defun lmerge (l)
      (if (null l)
          nil
          (merge (car l) (lmerge (cdr l)))))
```

4)

```
i) (defun hours (hms) (car hms))

    (defun mins (hms) (car (cdr hms)))

    (defun secs (hms) (car (cdr (cdr hms))))

    (defun too_secs (hms)
      (+ (* 60 (+ (* 60 (hours hms)) (mins hms))) (secs hms)))

    (defun from_secs (s)
      (list (truncate s 3600)
            (truncate (rem s 3600) 60)
            (rem s 60)))
```

```
ii) (defun tick (hms)
      (let ((h (hours hms))
            (m (mins hms))
            (s (secs hms)))
        (let ((s1 (+ s 1)))
          (if (< s1 60)
              (list h m s1)
              (let ((m1 (+ m 1)))
                (if (< m1 60)
                    (list h m1 0)
                    (let ((h1 (+ h 1)))
                      (if (< h1 24)
                          (list h1 0 0)
                          (list 0 0 0))))))))))
```

```
iii) (defun hms (trans) (car (cdr trans)))
```

```
(defun tless (tr1 tr2)
  (let ((t1 (hms tr1))
        (t2 (hms tr2)))
    (let ((h1 (hours t1))
          (m1 (mins t1))
          (s1 (secs t1))
          (h2 (hours t2))
          (m2 (mins t2))
          (s2 (secs t2)))
      (if (< h1 h2)
          t
          (if (= h1 h2)
              (if (< m1 m2)
                  t
                  (if (< s1 s2)
                      t
                      (list h1 m1 s1)))
              (list h1 m1 s1))))))
```

```
(if (= m1 m2)
    (if (< s1 s2)
        t
        nil)
    nil))
nil))))))
```

```
(defun tinsert (tr l)
  (cond ((null l) (cons tr nil))
        ((tless tr (car l)) (cons tr l))
        (t (cons (car l) (tinsert tr (cdr l))))))
```

```
(defun tsort (l)
  (if (null l)
      l
      (tinsert (car l) (tsort (cdr l)))))
```

5)

```
i) (defun tcomp (t1 t2)
    (cond ((and (null t1) (null t2)) t)
          ((or (null t1) (null t2)) nil)
          ((= (item t1) (item t2)) (and (tcomp (left t1) (left t2))
                                         (tcomp (right t1) (right t2))))
          (t nil)))
```

```
ii) (defun tfind (t1 t2)
    (cond ((null t1) t)
          ((null t2) nil)
          ((tcomp t1 t2) t)
          ((< (item t1) (item t2)) (tfind t1 (left t2)))
          (t (tfind t1 (right t2)))))
```

```
iii) (defun dtraverse (tree)
    (if (null tree)
        nil
        (append (dtraverse (right tree))
                  (cons (item tree) (dtraverse (left tree))))))
```